

# **FPGA based denoising using CNN**

A Major Project Work

Submitted in partial fulfilment of the requirements for the award of the  
Degree of

**BACHELOR OF TECHNOLOGY**

**IN**

**ELECTRONICS AND COMMUNICATION ENGINEERING**

By

K. S.G.H. BINDU

22EG104B49

Under the guidance of

**Mr. G. SREENIVASA RAJU, (Ph.D)**

Assistant Professor

Department of ECE



Department of Electronics and Communication Engineering

**ANURAG UNIVERSITY**

**SCHOOL OF ENGINEERING**

Venkatapur(V), Ghatkesar(M), Medchal-Malkajgiri Dist-500088

2025-2026

**ANURAG UNIVERSITY**  
**SCHOOL OF ENGINEERING**

Venkatapur(V), Ghatkesar(M), Medchal-Malkajgiri Dist-500088

DEPARTMENT OF ELECTRONICS AND COMMUNICATION  
ENGINEERING



**CERTIFICATE**

This is to certify that the project report entitled “**FPGA based denoising using CNN**” being submitted by

K. S.G.H. BINDU    -    22EG104B49

in partial fulfillment for the award of the Degree of Bachelor of Technology in Electronics & Communication Engineering to the Anurag University, Hyderabad is a record of bonafide work carried out under my guidance and supervision. The results embodied in this project report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

Mr. G. Sreenivasa Raju  
Assistant Professor  
Department of ECE

Dr. Harikrishna Kamatham  
Head of the Department  
DEPT OF ECE

External Examiner

## ACKNOWLEDGEMENT

This project is an acknowledgement to the inspiration, drive and technical assistance contributed by many individuals. This project would have never seen light of this day without the help and guidance I have received. I would like to express my gratitude to all the people behind the screen who helped me to transform an idea into a real application.

It's my privilege and pleasure to express my profound sense of gratitude to **Mr. G. Sreenivasa Raju, Assistant Professor**, Department of ECE for his guidance throughout this dissertation work. I express my sincere gratitude to **Dr. Harikrishna Kamatham, Professor, Head of Department**, Electronics and Communication Engineering for his precious suggestions for the successful completion of this project. He is also a great source of inspiration to our work.

I would like to express my deep sense of gratitude to **Dr. V. Vijaya Kumar, Dean School of Engineering** Anurag University for his tremendous support, encouragement and inspiration. Lastly, I thank almighty, our parents, friends for their constant encouragement without which this assignment would not be possible. I would like to thank all the other staff members, both teaching and non- teaching, which have extended their timely help and eased my work.

**BY**

K. S.G.H. BINDU

22EG104B49

## DECLARATION

I hereby declare that the result embodied in this project report entitled “**FPGA based denoising using CNN**” is carried out by me during the year 2025-2026 for the partial fulfilment of the award of **Bachelor of Technology in Electronics and Communication Engineering**, from ANURAG UNIVERSITY. I have not submitted this project report to any other Universities / Institute for the award of any degree.

**BY**

K. S.G.H. BINDU

22EG104B49

## **ABSTRACT**

The project presents the design and implementation of a Convolutional Neural Network (CNN)-based ECG signal denoising system on FPGA. The system integrates machine learning with hardware acceleration to achieve real-time, low-latency signal processing. Initially, ECG signals are corrupted with realistic noise components including powerline interference, baseline wander, and EMG noise. A CNN model is trained using PyTorch to learn noise removal patterns. The trained model weights are then converted into fixed-point Q1.15 format and deployed on an FPGA using Verilog. The hardware implementation includes multiple convolution layers, ReLU activations, buffering stages, and secure XOR-based encryption. Experimental results demonstrate that the CNN-based approach significantly outperforms traditional filtering techniques in terms of Signal-to-Noise Ratio (SNR) and Mean Squared Error (MSE).

# TABLE OF CONTENTS

| CHAPTER NO       | DESCRIPTION                               | PAGE NO      |
|------------------|-------------------------------------------|--------------|
|                  | <b>CERTIFICATE</b>                        | <b>i</b>     |
|                  | <b>ACKNOWLEDGEMENT</b>                    | <b>ii</b>    |
|                  | <b>DECLARATION</b>                        | <b>iii</b>   |
|                  | <b>ABSTRACT</b>                           | <b>iv</b>    |
|                  | <b>LIST OF FIGURES</b>                    | <b>vii</b>   |
|                  | <b>LIST OF TABLES</b>                     | <b>viii</b>  |
| <b>CHAPTER 1</b> | <b>INTRODUCTION</b>                       | <b>01-04</b> |
| 1.1              | Introduction                              | 01-02        |
| 1.2              | Problem Statement                         | 02           |
| 1.3              | Objectives                                | 02-04        |
| <b>CHAPTER 2</b> | <b>LITERATURE REVIEW</b>                  | <b>05-08</b> |
| <b>CHAPTER 3</b> | <b>METHODOLOGY</b>                        | <b>09-18</b> |
| 3.1              | System Overview                           | 09-10        |
| 3.2              | Block Diagram                             | 10-12        |
| 3.3              | Flowchart                                 | 12-13        |
| 3.4              | Algorithm                                 | 13-16        |
| 3.5              | System Architecture                       | 16-19        |
| 3.5.1            | Module: xor_encrypt                       | 16           |
| 3.5.2            | Top Module: cnn_top                       | 17           |
| 3.5.3            | Simulation Module: tb_cnn                 | 17-18        |
| 3.5.4            | Module : Conv_id                          | 18-19        |
| <b>CHAPTER 4</b> | <b>SOFTWARE&amp; HARDWARE DESCRIPTION</b> | <b>20-31</b> |
| 4.1              | Hardware Description                      | 20-23        |
| 4.1.1            | Key Features of Artix-7                   | 21           |
| 4.1.2            | Hardware development flow with Artix-7    | 21-22        |
| 4.1.3            | Example use case on Artix-7               | 23           |
| 4.1.4            | Advantages & Disadvantages of Artix-7     | 23           |
| 4.2              | Software Description                      | 23-27        |
| 4.2.1            | Basic Building Blocks                     | 24-25        |
| 4.2.2            | Vivado Files                              | 25           |

|                   |                                      |                |
|-------------------|--------------------------------------|----------------|
| 4.2.3             | Common Vivado Tasks and Commands     | 25             |
| 4.2.4             | Starting Vivado Software             | 25             |
| 4.2.5             | Creating a new project               | 26             |
| 4.2.6             | Adding source files                  | 26             |
| 4.2.7             | Adding constraints files             | 26             |
| 4.2.8             | Loading weights                      | 26-27          |
| 4.2.9             | running synthesis and implementation | 27             |
| 4.2.10            | Programming the FPGA hardware        | 27             |
| 4.2.11            | Method of Implementation             | 27             |
| 4.3               | VS code                              | 27-31          |
| 4.3.1             | Py torch Library                     | 27-28          |
| 4.3.2             | NumPy and SciPy                      | 28             |
| 4.3.3             | MatplotLib                           | 28             |
| 4.4               | Advantages And Disadvantages         | 31             |
| <b>CHAPTER 5</b>  | <b>RESULTS AND DISCUSSION</b>        | <b>32-40</b>   |
| 5.1               | Performance Metrics                  | 38             |
| 5.1.1             | Signal-to-Noise Ratio                | 38             |
| 5.1.2             | Mean Square Error                    | 38             |
| 5.1.3             | Observations                         | 38             |
| 5.2               | Performance Comparison               | 39             |
| 5.3               | Discussion                           | 39             |
| 5.4               | Limitations of Proposed System       | 40             |
| <b>CHAPTER 6</b>  | <b>CONCLUSION &amp; FUTURE SCOPE</b> | <b>41-43</b>   |
| 6.1               | Conclusion                           | 41             |
| 6.2               | Advantages                           | 41-42          |
| 6.3               | Future Scope                         | 42-43          |
| <b>REFERENCES</b> |                                      | <b>ix-x</b>    |
| <b>APPENDIX</b>   | Source Code                          | <b>xi-xxii</b> |

## LIST OF FIGURES

| FIGURE NO | NAME OF FIGURE                                         | PAGE NO |
|-----------|--------------------------------------------------------|---------|
| 3.1       | Block Diagram of signal using CNN                      | 10      |
| 3.2       | Flow chart of Signal Denoising using CNN               | 12      |
| 4.1       | FPGA Kit                                               | 20      |
| 4.2       | CNN Model Architecture using Pytorch                   | 28      |
| 4.3       | HDL Code editor                                        | 29      |
| 4.4       | Waveform simulation output                             | 29      |
| 4.5       | FPGA Layout view                                       | 30      |
| 5.1       | Signals generated using Python                         | 32      |
| 5.2       | Schematic diagram of Convolution & Relu                | 33      |
| 5.3       | Stimulation Waveform of ECG with Conv & Relu           | 34      |
| 5.4       | IP Integrator Block                                    | 35      |
| 5.5       | IP Integrator Block of 4 Convolution layers            | 36      |
| 5.6       | Hardware Implementation Result                         | 36      |
| 5.7       | Hardware Implementation Result of 4 Convolution layers | 37      |



## **LIST OF TABLES**

| <b>TABLE NO</b> | <b>NAME OF THE TABLE</b>               | <b>PAGE NO</b> |
|-----------------|----------------------------------------|----------------|
| 2.1             | Table 2.1 Summary of Literature Survey | 08             |
| 5.1             | Table 5.1 Performance Comparison       | 39             |

# CHAPTER 1

## INTRODUCTION

### 1.1 INTRODUCTION

Electrocardiogram (ECG) signals play a crucial role in the diagnosis and monitoring of cardiovascular diseases. These signals provide valuable information about the electrical activity of the heart and are widely used in clinical and wearable healthcare systems. However, ECG signals are highly susceptible to various types of noise during acquisition, transmission, and processing. Common noise sources include powerline interference (50/60 Hz), baseline wander, and electromyographic (EMG) noise, all of which can significantly degrade signal quality and affect diagnostic accuracy.

Traditional signal processing techniques, such as notch filters and bandpass filters, have been extensively used for ECG denoising. While these methods are effective for removing specific types of noise, they often struggle to handle complex and non-stationary noise patterns. Additionally, conventional filtering approaches may distort important features of the ECG waveform, such as the P-wave, QRS complex, and T-wave. In recent years, deep learning techniques—particularly Convolutional Neural Networks (CNNs) have emerged as powerful tools for signal processing applications. CNNs are capable of learning complex nonlinear relationships and extracting meaningful features directly from raw data. This makes them highly suitable for biomedical signal denoising tasks, where noise characteristics can vary significantly.

This project focuses on the design and implementation of a CNN-based ECG denoising system on an FPGA (Field Programmable Gate Array). The primary objective is to leverage the high accuracy of deep learning models along with the real-time processing capability of FPGA hardware. The CNN model is first trained in software using noisy ECG signals, and the trained weights are then converted into a fixed-point format suitable for hardware implementation. The proposed system integrates multiple components, including convolution layers, activation functions (ReLU), buffering mechanisms, and a lightweight encryption scheme for secure data handling. The complete design is implemented using Verilog and validated through simulation and real-time hardware testing.

By combining machine learning with hardware acceleration, this work aims to provide a high-performance, low-latency, and efficient solution for ECG signal denoising, making it suitable for real-time healthcare and embedded biomedical applications.

## **1.2 PROBLEM STATEMENT**

Electrocardiogram (ECG) signals are widely used for monitoring and diagnosing cardiovascular conditions. However, during signal acquisition and transmission, ECG signals are often corrupted by various types of noise such as powerline interference (50/60 Hz), baseline wander, and electromyographic (EMG) noise. These noise components degrade signal quality and can obscure important features of the ECG waveform, leading to inaccurate analysis and diagnosis.

Conventional signal processing techniques, including notch filters and bandpass filters, are commonly used for ECG denoising. Although these methods are effective for removing specific types of noise, they are limited in their ability to handle complex and non-stationary noise patterns. In addition, traditional filtering techniques may distort essential components of the ECG signal, such as the QRS complex, which is critical for clinical interpretation.

With the growing demand for efficient and high-accuracy signal processing systems, there is a need for advanced denoising techniques that can effectively remove noise while preserving important signal characteristics. Deep learning models, particularly Convolutional Neural Networks (CNNs), have demonstrated superior performance in handling complex signal patterns. However, implementing such models directly in software can be computationally intensive.

Therefore, the problem addressed in this project is to design and implement an efficient ECG denoising system using a Convolutional Neural Network (CNN) on FPGA hardware. The proposed system aims to improve signal quality by effectively removing noise while preserving important ECG features, while also ensuring efficient hardware utilization.

## **1.3 OBJECTIVE**

The main objective of this project is to design and implement an efficient ECG signal denoising system using a Convolutional Neural Network (CNN) on FPGA hardware.

## 1. To Develop a Signal Denoising System

- To acquire and preprocess input signals for analysis.
- To normalize the signal to ensure stable processing.
- To prepare the signal for further noise addition and training.

## 2. To Simulate Noise Conditions

- To introduce different types of noise such as powerline interference, baseline wander, and EMG noise.
- To generate noisy signals that resemble real-world conditions.
- To study the effect of noise on signal quality.

## 3. To Design and Train a CNN Model

- To develop a Convolutional Neural Network (CNN) for signal denoising.
- To train the model using noisy input and clean target signals.
- To enable the network to learn the mapping between noisy and clean signals.

## 4. To Evaluate Model Performance

- To measure the performance using metrics such as Signal-to-Noise Ratio (SNR) and Mean Squared Error (MSE).
- To compare CNN results with traditional filtering methods.
- To validate the effectiveness of the denoising model.

## 5. To Implement Quantization for Hardware

- To convert floating-point weights into fixed-point (Q1.15 format).
- To reduce hardware complexity and improve efficiency.
- To ensure compatibility with FPGA implementation.

#### 6. To Incorporate Data Security

- To design an XOR-based encryption module for input signal protection.
- To encrypt the signal before processing and decrypt after processing.
- To ensure secure handling of sensitive data.

#### 7. To Implement the System on FPGA

- To synthesize and implement the design using FPGA tools such as Vivado.
- To integrate all modules into a complete system.
- To validate the functionality on hardware.

#### 8. To Verify the System Performance

- To perform simulation using waveform analysis tools.
- To test the system using hardware debugging tools such as VIO.
- To compare input and output signals to confirm noise removal.

#### 9. To Achieve Efficient Signal Denoising

- To remove noise while preserving important signal features.
- To improve signal quality compared to traditional methods.
- To demonstrate the effectiveness of CNN-based denoising on FPGA.

## CHAPTER 2

### LITERATURE SURVEY

**Chauhan & Changlani [1]** presented on Signal Processing in VLSI Circuits Using Neural Networks. Their work focuses on the implementation of neural network-based signal processing within VLSI circuits. The authors demonstrate how neural networks can be integrated into hardware systems to improve processing efficiency and accuracy. The study highlights the advantages of using neural architectures for handling complex signal operations while maintaining compatibility with hardware design. However, the approach introduces design complexity and requires training, which increases resource utilization in VLSI systems.

**Thippeswamy et al. [2]** introduced on AI Based Machine Learning Algorithm in VLSI Technology. The authors present methods to enhance system performance using intelligent algorithms that automate signal processing tasks. The approach improves scalability and efficiency but introduces challenges such as increased hardware complexity and the need for extensive training datasets. The study emphasizes the growing importance of AI integration in hardware design.

**Reddy et al. [3]** presented on AI-Driven Signal Processing for Communication Systems. Machine learning models are used to reduce noise and enhance signal quality, resulting in better communication reliability. The proposed method demonstrates adaptability to varying noise conditions. However, the approach requires significant computational resources and large datasets, making it less suitable for direct hardware implementation without optimization.

**Mashhadi et al. [4]** proposed a Deep Learning Denoising for EEG Signals. The model effectively distinguishes between noise and useful signal components, resulting in improved signal clarity. The study demonstrates the robustness of deep learning in biomedical signal processing. However, the model is computationally intensive and requires large amounts of training data, which can limit its practical implementation.

**Roy et al. [5]** focused the use of deep convolutional neural networks for image denoising. The CNN model effectively extracts features and removes noise while preserving image quality. The results show superior performance compared to traditional methods. However, the model involves high computational complexity and requires significant

training time and memory resources.

**Shekhar et al. [6]** presents a CNN-based approach for signal denoising in wearable systems. The model is designed to improve signal quality while maintaining efficiency for wearable applications. The results demonstrate accurate denoising and suitability for embedded systems. However, hardware implementation requires optimization due to resource constraints, and scalability remains a challenge.

**Liu et al. [7]** focused on CNN-based digital signal denoising techniques. The authors demonstrate how convolutional neural networks can effectively remove noise from signals while maintaining important features. The method provides high accuracy and reliability compared to traditional approaches. However, it requires significant hardware resources and careful optimization for efficient implementation.

**Shukla & Izharuddin [8]** proposed a hybrid deep learning model for signal denoising. By combining multiple deep learning techniques, the model achieves improved accuracy and better signal reconstruction. The approach is robust and effective in handling complex noise patterns. However, it introduces increased complexity, higher training costs, and challenges in hardware implementation.

| S.No | Journal name/ year         | Title                                                    | Methodology                                        | Advantages                                                                  | Limitations                                                    |
|------|----------------------------|----------------------------------------------------------|----------------------------------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------|
| 1    | Chauhan & Changlani (2025) | Signal Processing in VLSI Circuits Using Neural Networks | Used neural networks in VLSI for signal processing | 1. Efficient processing<br>2. Improved accuracy<br>3. Suitable for hardware | 1. Complex design<br>2. Requires training<br>3. Resource usage |
| 2    | Thippeswamy et al. (2023)  | AI Based Machine Learning Algorithm in VLSI Technology   | Applied ML algorithms in VLSI systems              | 1. High performance<br>2. Automation<br>3. Scalable                         | 1. High complexity<br>2. Training required<br>3. Hardware cost |

|   |                        |                                                       |                                            |                                                                   |                                                                        |
|---|------------------------|-------------------------------------------------------|--------------------------------------------|-------------------------------------------------------------------|------------------------------------------------------------------------|
| 3 | Reddy et al. (2022)    | AI-Driven Signal Processing for Communication Systems | Used ML for noise reduction in signals     | 1. Better noise removal 2. Improved communication 3. Adaptive     | 1. Computational cost 2. Requires dataset 3. Training time             |
| 4 | Mashhadi et al. (2020) | Deep Learning Denoising for EEG Signals               | Applied deep learning for artifact removal | 1. High accuracy 2. Effective noise removal 3. Robust             | 1. Complex model 2. High training cost 3. Data dependency              |
| 5 | Roy et al. (2021)      | Image Denoising using Deep CNN Techniques             | Used CNN for image denoising               | 1. High quality output 2. Good feature extraction 3. Efficient    | 1. High computation 2. Large model size 3. Training needed             |
| 6 | Shekhar et al. (2021)  | CNN-Based ECG Denoising in Wearable Systems           | CNN model for ECG noise removal            | 1. Accurate results 2. Suitable for wearable systems 3. Efficient | 1. Hardware complexity 2. Requires optimization 3. Limited scalability |
| 7 | Liu et al. (2024)      | CNN-Based Digital Signal Denoising Techniques         | Used CNN for signal denoising              | 1. High performance 2. Better accuracy 3. Reliable                | 1. Resource intensive 2. Training required 3. Complex design           |



|   |                            |                                        |                                             |                                                         |                                                             |
|---|----------------------------|----------------------------------------|---------------------------------------------|---------------------------------------------------------|-------------------------------------------------------------|
| 8 | Shukla & Izharuddin (2023) | Hybrid Deep Learning for ECG Denoising | Combined deep learning models for denoising | 1. Improved accuracy 2. Better reconstruction 3. Robust | 1. High complexity 2. Training cost 3. Hardware limitations |
|---|----------------------------|----------------------------------------|---------------------------------------------|---------------------------------------------------------|-------------------------------------------------------------|

**Table 2.1 Summary of Literature Survey**

## **CHAPTER 3**

### **METHODOLOGY**

#### **3.1 SYSTEM OVERVIEW**

The proposed system presents a hybrid approach that combines deep learning with hardware acceleration for signal denoising. The system is designed to efficiently remove noise while preserving essential features required for accurate analysis.

The input signals are acquired from the MIT-BIH Arrhythmia dataset, which contains real biomedical data. These signals are first normalized to ensure consistency and stability during processing. To simulate practical conditions, different types of noise such as powerline interference (50 Hz), baseline wander, and electromyographic (EMG) noise are added to the signals.

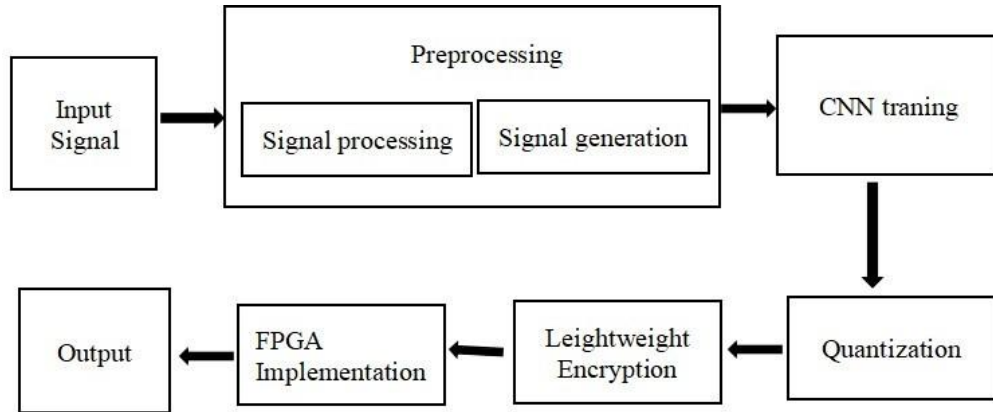
A Convolutional Neural Network (CNN) is then designed and trained using Python and PyTorch. The CNN architecture consists of multiple one-dimensional convolution layers that perform feature extraction and noise removal. During training, noisy signals are provided as input and clean signals as the target output. The model learns the mapping between noisy and clean signals using backpropagation and optimization techniques. The performance of the model is evaluated using metrics such as Signal-to-Noise Ratio (SNR) and Mean Squared Error (MSE).

After successful training, the learned parameters, including weights and biases, are converted from floating-point representation to fixed-point (Q1.15 format). This conversion is essential for hardware implementation, as FPGA devices are more efficient in handling fixed-point arithmetic. The use of fixed-point representation reduces hardware complexity and improves computational efficiency.

The hardware implementation is carried out using Verilog HDL. The CNN architecture is mapped onto FPGA using modules such as input buffers, convolution units, activation functions (ReLU), and control logic. The input buffer generates a sliding window of data samples, which are processed through multiple convolution and activation layers in a sequential manner. The design follows a pipelined architecture to ensure efficient data flow across all stages.

Additionally, a lightweight XOR-based encryption mechanism is incorporated to provide secure data handling. The input signal is first encrypted, then processed through the CNN layers, and finally decrypted to produce the denoised output signal.

### 3.2 BLOCK DIAGRAM



**Fig 3.1 Block Diagram of signal denoising using CNN**

#### ECG Input

The ECG input block represents the initial stage of the system where the raw electrocardiogram signal is obtained. In this project, the ECG signal is taken from the MIT-BIH dataset, which provides real-world biomedical data. The signal is sampled and digitized to make it suitable for digital processing.

To simulate real-world conditions, different types of noise are artificially added to the clean ECG signal. Three major types of noise are considered in this project. The first is powerline interference, which is generated as a sinusoidal signal at 50 Hz. The second is baseline wander, modeled as a low-frequency sinusoidal component at 0.5 Hz, representing slow variations in the signal baseline. The third is high-frequency noise (EMG noise), generated as a sinusoidal component at 120 Hz.

These noise components are combined with the original ECG signal to produce a noisy signal. This noisy signal is used as the input to the CNN model, while the original clean signal serves as the target output.

#### Preprocessing

This signal is first passed to the preprocessing stage, where it undergoes two important operations: signal processing and signal generation. In signal processing, the input data is

normalized and prepared for further analysis. In signal generation, noise components such as powerline interference, baseline wander, and other disturbances are added to simulate real-world conditions. The preprocessed signal is then provided to the CNN training block. In this stage, a Convolutional Neural Network (CNN) is trained using noisy input signals and corresponding clean signals. The network learns to identify noise patterns and reconstruct the original signal using optimization techniques.

### **CNN Denoising**

The CNN model was then trained using noisy input signals and corresponding clean signals. The training process showed a reduction in loss values across epochs, confirming that the model successfully learned the mapping between noisy and clean signals. After training, the CNN achieved a significantly improved SNR of 15.89 dB and a reduced MSE of 0.0030, demonstrating superior denoising performance compared to traditional methods.

### **Quantization**

The trained model parameters were converted into fixed-point to enable efficient FPGA implementation. The quantized weights were successfully integrated into the hardware design without significant loss in accuracy. The FPGA implementation was carried out using Verilog, where different modules such as input buffer, convolution layers, activation units, and control logic were integrated.

### **Encryption Module**

The encryption module was implemented using an XOR-based technique to secure the signal before processing. This method ensured data protection with minimal hardware overhead. The encrypted signal was then processed through the CNN architecture. The input buffer efficiently managed. In this project, a simple XOR-based encryption technique is used due to its low hardware complexity. The input signal is combined with a predefined key to generate an encrypted output. This ensures that sensitive medical data is protected during processing. The encryption module adds an extra layer of security without affecting system performance.

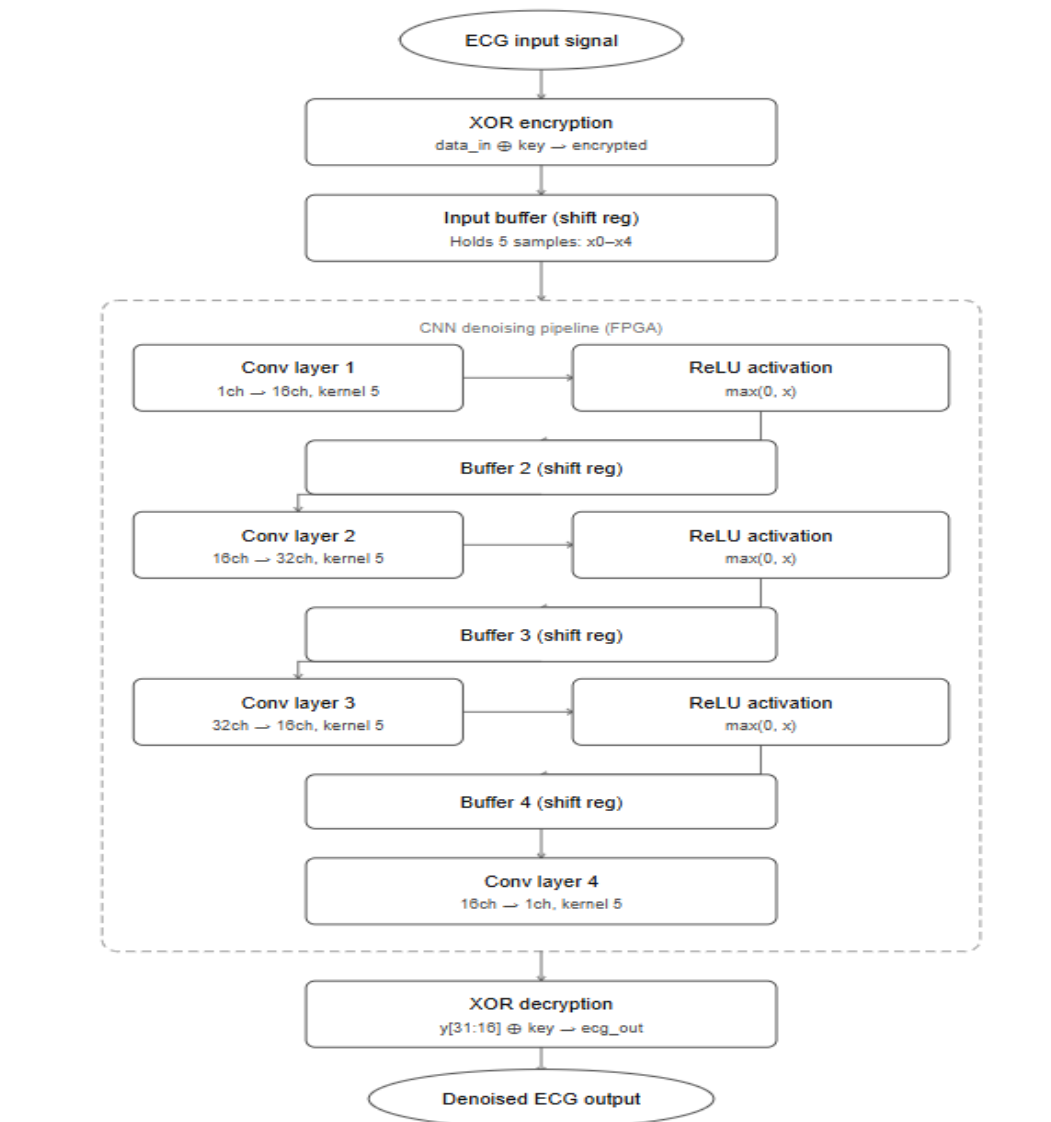
### **FPGA Implementation**

After encryption, the system proceeds to the FPGA implementation block. In this stage, the CNN architecture is implemented using Verilog HDL. The design includes modules such as input buffers, convolution units, and activation functions. The system processes the input data through multiple layers in a structured manner to remove noise effectively.

## Output

The output block provides the final denoised ECG signal after processing through all stages. This signal is free from noise and retains important diagnostic features. The output can be observed using tools like Vivado VIO or displayed using external devices. This block confirms the effectiveness of the proposed system. The improved signal quality demonstrates the success of CNN-based denoising on FPGA. The processed signal is obtained at the output block, which represents the denoised signal with improved quality and reduced noise.

## 3.3 FLOWCHART



**Fig 3.2 Flow Chart of signal denoising using CNN**

The process begins with the input signal, which serves as the raw data for the system. This signal is first passed through an XOR encryption block, where it is combined with a

predefined key to generate an encrypted version of the signal. This ensures secure handling of the data during processing.

The encrypted signal is then provided to the input buffer, which acts as a shift register. The buffer stores sequential samples and forms a sliding window consisting of five samples ( $x_0$ – $x_4$ ). This window is required for performing convolution operations in the CNN layers.

The buffered data enters the CNN denoising pipeline, which consists of multiple convolution and activation stages. In the first stage, Convolution Layer 1 processes the input using a kernel of size 5 and multiple filters. The output is then passed through a ReLU activation function, which eliminates negative values and introduces non-linearity.

The output of each convolution layer is passed through intermediate buffer stages. These buffers maintain data continuity and prepare the input for the next convolution stage. This process is repeated across multiple layers, including Convolution Layer 2, Convolution Layer 3, and Convolution Layer 4, with each layer progressively refining the signal and reducing noise.

After processing through all CNN layers, the signal is passed to the XOR decryption block, where the same key used during encryption is applied to recover the original data structure. This ensures that the final output is both secure and accurate.

Finally, the system produces the denoised output signal, which has reduced noise while preserving important features. This flow demonstrates the complete pipeline, including encryption, buffering, convolution, activation, and decryption, resulting in an efficient and structured denoising system.

### **3.4 ALGORITHM**

The proposed algorithm describes the step-by-step process for ECG signal denoising using a Convolutional Neural Network (CNN) implemented on FPGA. The algorithm includes both software-based training and hardware-based execution stages.

#### **Step 1: ECG Signal Acquisition**

The ECG signal  $x(n)$  is obtained from the MIT-BIH dataset. The signal is sampled and converted into digital form suitable for processing. This signal represents the original clean biomedical data.

#### **Step 2: Signal Normalization**

The acquired ECG signal is normalized to bring all values within a specific range. This ensures stability during training and hardware implementation.

$$x_{norm}(n) = \frac{x(n)}{\max(|x(n)|)}$$

Normalization helps in improving convergence during CNN training.

### Step 3: Noise Addition

Artificial noise is added to simulate real-world conditions. The noisy signal is represented as:

$$y(n) = x(n) + n(n)$$

Where:

- $n(n)$  includes:
- Powerline noise (50 Hz)
- Baseline wander
- EMG noise

### Step 4: CNN Model Training

A CNN model is trained using noisy signals as input and clean signals as target output. The model consists of multiple convolution layers followed by ReLU activation.

The loss function used is Mean Squared Error (MSE):

$$MSE = \frac{1}{N} \sum (y_{true} - y_{pred})^2$$

The model learns to minimize noise and preserve signal features.

### Step 5: Weight Extraction

After training, the weights and biases of the CNN model are extracted. These parameters represent the learned features required for denoising.

### Step 6: Fixed-Point Conversion

The floating-point weights are converted into fixed-point format (Q1.15) to make them compatible with FPGA implementation.

$$Fixed = Round(Float \times 2^{15})$$

This reduces hardware complexity and improves processing speed

### Step 7: FPGA Implementation

The CNN model is implemented on FPGA using Verilog HDL. The architecture includes:

- Input buffer
- Convolution modules
- ReLU activation
- Control logic

The FPGA processes data in parallel, enabling real-time performance.

### Step 8: Input Signal Injection

The noisy ECG signal is provided to FPGA using VIO (Virtual Input/Output). This allows real-time testing without external hardware.

### Step 9: Encryption

The input signal is encrypted using XOR logic:

$$y_e(n) = y(n) \oplus key$$

This ensures secure processing of medical data.

### Step 10: Convolution Operation

The convolution operation is applied using sliding window technique:

$$z(n) = \sum_{i=0} w_i \cdot x(n - i)$$

This extracts important features and reduces noise.

### Step 11: Activation Function (ReLU)

The ReLU function is applied to introduce non-linearity:

$$f(z) = \max(0, z)$$

Negative values are removed, improving signal clarity.

### Step 12: Multi-Layer Processing

Steps 10 and 11 are repeated across multiple CNN layers to progressively refine the signal.



### Step 13: Decryption

The processed signal is decrypted using the same key:

$$\text{output} = f(z) \oplus \text{key}$$

### Step 14: Output Generation

The final output is a denoised ECG signal with reduced noise and preserved features. This output is observed using Vivado VIO.

## 3.5 SYSTEM ARCHITECTURE

This section describes the internal architecture of each hardware module in which each module performs a specific function and interacts with other modules to achieve overall signal denoising. The interaction between these modules ensures a smooth and continuous flow of data from input to output.

### 3.5.1 Module : xor\_encrypt

The encryption module is responsible for securing the input signal before it is processed by the system. In this project, a lightweight XOR-based encryption technique is used due to its simplicity and low hardware complexity.

This method is well-suited for FPGA implementation, as it requires minimal resources and does not introduce significant computational overhead. The encryption process involves performing a bitwise XOR operation between the input signal and a predefined key.

The mathematical representation of this operation is given by:

$$\text{Encrypted Signal} = \text{Input Signal} \oplus \text{Key}$$

This operation transforms the original signal into an encoded form, ensuring that the data remains protected during processing. The use of a fixed key allows for consistent encryption and decryption within the system.

```
module xor_encrypt(  
    input [15:0] data_in,  
    input [15:0] key,  
    output [15:0] data_out  
);  
  
assign data_out = data_in ^ key;  
  
endmodule
```

### **3.5.2 Top Module: cnn\_top**

The `cnn_top` module is the top-level module of the system that integrates all the functional blocks required for signal denoising. It acts as the central unit that manages data flow between different modules such as encryption, input buffering, convolution layers, activation functions, and decryption.

The module receives the input signal, clock signal, and encryption key as inputs. Initially, the input signal is passed through the XOR-based encryption module. The encrypted signal is then fed into the input buffer, which generates a sliding window of samples required for convolution operations.

The buffered data is processed through multiple convolution and ReLU layers in a sequential manner. Intermediate buffer stages are used between layers to ensure proper data alignment and synchronization.

After processing through all layers, the output is passed through the decryption module to recover the final denoised signal.

### **3.5.3 Simulation Module: tb\_cnn**

The `tb_cnn` module is used for simulation and verification of the CNN-based denoising system. It provides the necessary input signals and test conditions to evaluate the functionality of the design.

The testbench generates a clock signal using a periodic toggle mechanism and applies different input signal values at specified time intervals. It also provides an encryption key to the system. These inputs are fed into the `cnn_top` module, which acts as the Device Under Test (DUT).

The purpose of the testbench is to observe the output behavior of the system under different input conditions. It helps in verifying whether the design performs correctly and produces the expected denoised output.

```

module tb_cnn;
reg clk;
reg [15:0] ecg_in;
reg [15:0] key;
wire [15:0] ecg_out;
cnn_top DUT(
.clk(clk),
.ecg_in(ecg_in),
.key(key),
.ecg_out(ecg_out)
);
/* clock generation */
initial
begin
clk=0;
forever #5 clk=~clk;
end /* stimulus */
initial begin
key = 16'h55AA;
ecg_in = 0;
#100 ecg_in = 120;
#100 ecg_in = 210;
#100 ecg_in = 180;
#100 ecg_in = 320;
#100 ecg_in = 280;
#100 ecg_in = 420;
#100 ecg_in = 360;
#200 $finish;
end
endmodule

```

### 3.5.4 Module conv1d

The convolution module is the fundamental processing unit of the CNN architecture and is responsible for performing feature extraction from the input signal. It implements a one-dimensional convolution operation using a fixed number of input samples and corresponding weights.

The convolution operation is represented a

$$y = x_0 \cdot w_0 + x_1 \cdot w_1 + x_2 \cdot w_2 + x_3 \cdot w_3 + x_4 \cdot w_4$$

```

module conv1d(
input signed [15:0] x0,x1,x2,x3,x4,
output signed [31:0] y
);

parameter signed w0 = 16'd3097;
parameter signed w1 = 16'd12185;
parameter signed w2 = -16'd4601;
parameter signed w3 = -16'd8618;
parameter signed w4 = -16'd5184;

assign y = x0*w0 + x1*w1 + x2*w2 + x3*w3 + x4*w4;

endmodule

```

The weights are stored in fixed-point format which allows efficient implementation on FPGA hardware. The output is generated with higher precision to avoid loss of information during computation. This module is used repeatedly across multiple layers in the CNN architecture, enabling progressive noise reduction and feature refinement at each stage

## CHAPTER 4

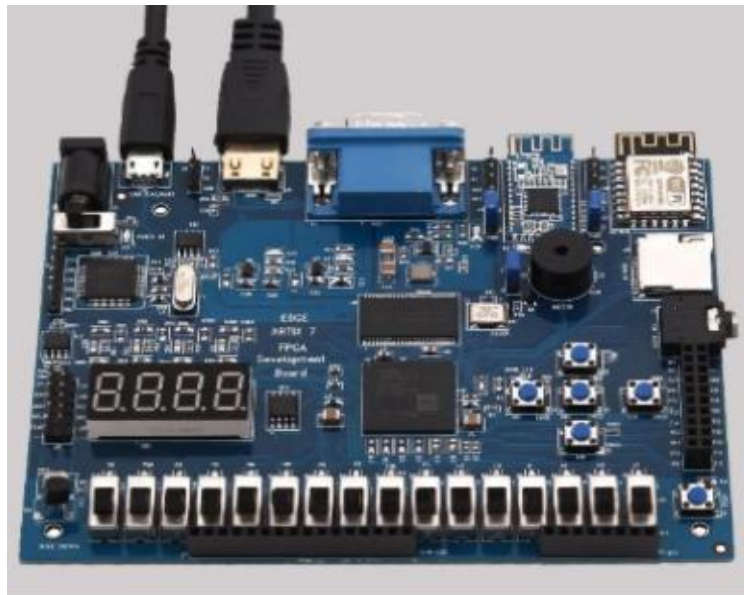
### HARDWARE AND SOFTWARE REQUIREMENTS

#### 4.1 HARDWARE DESCRIPTION

The Artix-7 family is a cost-effective, low-power series of FPGAs from Xilinx (now AMD), designed for high-performance logic implementations with an emphasis on efficiency and affordability. It offers a great balance of logic capacity, I/O performance, and power consumption, making it ideal for academic projects, signal processing, embedded systems, and communication applications.

Artix-7 FPGAs are commonly used in boards like:

- Basys 3 (xc7a35tcpg236-1)
- Nexys A7 (xc7a100t-1CSG324C)
- Custom boards with JTAG or USB programming interface



**Fig 4.1 FPGA Kit**

Fig 4.1 represents an FPGA Development Kit is a hardware platform centered around a Field Programmable Gate Array (FPGA) chip. It is used by engineers and students to design, test, and prototype custom digital circuits. The board typically includes the FPGA, memory, various I/O ports (like switches, LEDs, and connectors), and the necessary programming

interface to load the hardware description code. The kit allows for rapid prototyping and real-world testing of designs written in HDL (like VHDL or Verilog).

#### 4.1.1 Key Features of Artix-7:

| Feature             | Details                                                               |
|---------------------|-----------------------------------------------------------------------|
| Process Technology  | 28nm                                                                  |
| Logic Cells         | Up to 215K logic cells (varies by part)                               |
| I/O Pins            | Configurable, supports multiple I/O standards                         |
| DSP Slices          | For multiply-accumulate operations                                    |
| BRAM                | Block RAM available for internal memory                               |
| Clock Management    | MMCM and PLL for clock generation                                     |
| Power Consumption   | Low compared to previous generation Spartan-6                         |
| Interface Supported | SPI, UART, <a href="#">I2C</a> , <a href="#">VGA</a> , HDMI, Ethernet |
| Package             | e.g., CPG236, CSG324                                                  |

#### 4.1.2 Hardware Development Flow with Artix-7

This is a step-by-step guide for developing on Artix-7 using **Vivado**:

##### Step 1: Write the Verilog/VHDL Code

Create a module (example: blinking LED)

```

module blink_led (
    input clk,          // 100 MHz clock input
    output reg led);

    reg [25:0] counter = 0;
    always @(posedge clk) begin
        counter <= counter + 1;
        if(counter == 50000000) begin // 0.5s toggle at 100MHz
            counter <= 0;
            led <= ~led;
        end
    end
endmodule

```

```
        end
    end
endmodule
```

### **Step 2: Create XDC File (Pin Constraints)**

Example constraints.xdc (for Basys 3 board):

```
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
set_property PACKAGE_PIN U16 [get_ports led]
set_property IOSTANDARD LVCMOS33 [get_ports led]
```

Match these pin names with your board's manual. Each I/O pin must be correctly mapped.

### **Step 3: Vivado Project Steps**

- Create New Project
- Add Verilog Source File
- Add Constraints File (.xdc)
- Select FPGA Part (e.g., xc7a35tcpg236-1)
- Run Synthesis
- Run Implementation
- Generate Bitstream

### **Step 4: Program the FPGA**

- Open Hardware Manager
- Auto-connect to target FPGA
- Program Device with the generated bit file
- LED will start blinking based on your HDL logic.

### 4.1.3 Example Use Cases on Artix-7

| Application       | Description                                           |
|-------------------|-------------------------------------------------------|
| Image Processing  | Edge detection, grayscale conversion using FPGA logic |
| Signal Processing | FIR/IIR filter implementation                         |
| IoT               | UART/I2C interfaces with sensors                      |
| Robotics          | Motor control, encoder reading                        |
| AI (Basic)        | Neural inference using DSP blocks                     |
| Communication     | UART, SPI, VGA output                                 |

### 4.1.4 Advantages and Disadvantages of Artix-7

#### Advantages:

- Low cost for entry-level FPGA development
- Low power consumption
- Rich documentation and community support
- Great for learning FPGA development
- Flexible I/O and clocking resources

#### Disadvantages:

- Limited logic resources for large AI/ML models
- No ARM processor (unlike Zynq series)
- Slower than high-end FPGAs (e.g., Kintex, Virtex)
- Limited external memory support without additional components

## 4.2 SOFTWARE DESCRIPTION

In this project, we use Xilinx Vivado Design Suite software to design and implement digital circuits on FPGA (Field Programmable Gate Array). Vivado is a modern, high-performance Electronic Design Automation (EDA) tool developed by Xilinx (now AMD). It is used to design, simulate, synthesize, implement, generate bitstream, and program FPGA devices such as Artix-7.



Vivado supports HDL (Hardware Description Languages) such as:

- Verilog HDL (most commonly used)
- VHDL
- System Verilog

FPGA (Field Programmable Gate Array) is a reprogrammable digital hardware chip that allows developers to implement custom logic at the hardware level. Unlike microcontrollers (which execute instructions sequentially), FPGAs work on parallel hardware architecture, providing high performance and flexibility.

Vivado allows:

- RTL (Register Transfer Level) design entry
- HDL simulation and debugging
- Synthesis of design into logic gates
- Placement and routing on FPGA fabric
- Bitstream generation (.bit file)
- Programming the FPGA directly from Vivado

Vivado provides built-in IP cores, constraints management, timing analysis, memory generation, and debugging tools such as ILA (Integrated Logic Analyzer).

Just like MATLAB supports matrix operations, Vivado supports hardware design and verification at the circuit level. It replaces older tools like ISE Design Suite and provides better performance, 64-bit processing, and support for Artix-7, Kintex-7, Virtex-7, and Zynq devices.

#### **4.2.1 Basic Building Blocks**

Vivado operates in two main design flows:

- RTL-based (Textual Design) using Verilog/VHDL
- Block Design Flow (Graphical) using IP Integrator Vivado Windows:
  - Start Page: Initial window to create/open projects
- Project Manager: View hierarchy and manage files
- Design Sources: Your Verilog/VHDL files
- Simulation Sources: Testbenches for simulation
- IP Catalog: To add pre-built Intellectual Property (IP) blocks

- Tcl Console: Command line interface for automation
- Messages and Reports: Logs for synthesis, implementation, errors
- Example:  

```
module AND_Gate (input A, input B, output Y); assign Y = A & B;
endmodule
```

#### **4.2.2 Vivado Files**

- .xpr – Vivado project file
- .v / .vhdl – Verilog/VHDL source files
- .xdc – Xilinx Design Constraints file (used for pin mapping)
- .bit – Bitstream file used to program the FPGA
- .bin / .mcs – Binary or PROM file for flash memory configuration
- .jou, .log – Journal and log files for debugging

#### **4.2.3 Common Vivado Tasks and Commands**

- Command - Description
- create\_project - Starts a new Vivado project add\_files - Adds Verilog/VHDL source code
- set\_property - Assigns attributes to files or constraints synthesize\_design - Synthesizes HDL into gate-level netlist implement\_design - Maps netlist onto FPGA logic generate\_bitstream - Creates .bit file
- open\_hw\_target - Connects to FPGA hardware program\_device - Loads .bit file onto FPGA

#### **4.2.4 Starting the Vivado Software**

- Double-click on the Vivado icon to launch the IDE
- Once opened, the Start Page gives options to:
- Create a new project
- Open an existing project
- View recent projects

#### **4.2.5 Creating a New Project**

- Go to File > New Project
- Set the project name and location
- Choose the project type: RTL Project (Do not specify sources at this time)
- Add source files (Verilog/VHDL)
- Add constraint files (.xdc) for pin mapping
- Select the FPGA part number or board (e.g., xc7a35tcpg236-1 for Artix-7)

#### **4.2.6 Adding Source Files**

- Go to Project Manager > Add Sources
- Choose “Add or Create Design Sources”
- Browse or create new .v or .vhdl files
- Example file: and\_gate.v

#### **4.2.7 Adding Constraints File**

- Choose “Add Constraints”
- Browse for .xdc file or create a new one
- Map each I/O port in your HDL code to physical pins of the FPGA
- Example:  
set\_property PACKAGE\_PIN W5 [get\_ports A]

#### **4.2.8 Loading weights**

- CNN weights were converted from floating-point to fixed-point (Q1.15 format).
- The fixed-point weights were stored in a .coe (Coefficient) file.
- The .coe file contains weight values in a structured format for memory initialization.
- Block Memory Generator IP was used from the Vivado IP Catalog.
- The .coe file was loaded into BRAM during IP configuration.
- Weights are stored in FPGA memory for efficient access during convolution.
- Address lines are used to read weights sequentially.

- This approach reduces hardware complexity and saves FPGA resources.
- Weights can be easily updated by modifying the .coe file without changing Verilog code.

#### **4.2.9 Running Synthesis and Implementation**

- Click Run Synthesis
- After synthesis, click Run Implementation
- Fix any errors/warnings shown in the Messages window
- After successful implementation, click Generate Bitstream

#### **4.2.10 Programming the FPGA Hardware**

- Connect your FPGA board via USB/JTAG
- Open Hardware Manager > Open Target > Auto Connect
- Click Program Device and select your .bit file
- FPGA will be configured and ready to run

#### **4.2.11 Method of Implementation**

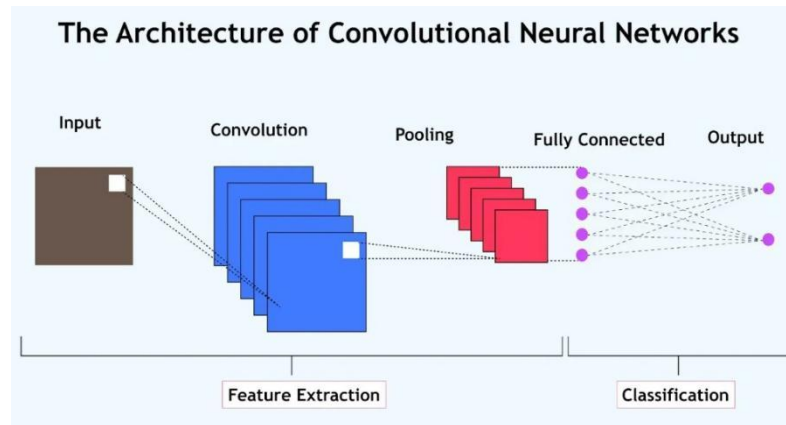
- Launch Vivado and create a new RTL project
- Add design files (.v) and constraints file (.xdc)
- Write your Verilog/VHDL code using the Text Editor
- Run Synthesis and Implementation
- Generate Bitstream (.bit) file
- Connect FPGA Board and open Hardware Manager
- Program the Device with the generated bitstream
- Test the functionality using external hardware (switches, LEDs, displays)
- Create a module (example: blinking LED)

### **4.3 VS Code**

#### **4.3.1 PyTorch Library**

PyTorch is a deep learning framework used to design and train the CNN model. It provides built-in functions for convolution layers, activation functions, and optimization algorithms.

The CNN model is trained using noisy ECG signals as input and clean signals as output. PyTorch enables efficient model training using backpropagation and gradient descent techniques.



**Fig 4.2 CNN Model Architecture using PyTorch**

#### 4.3.2 NumPy and SciPy

NumPy is used for numerical computations, while SciPy is used for signal processing operations such as filtering and noise generation. These libraries help in:

- Signal generalization and processing
- Noise addition
- Mathematical operations

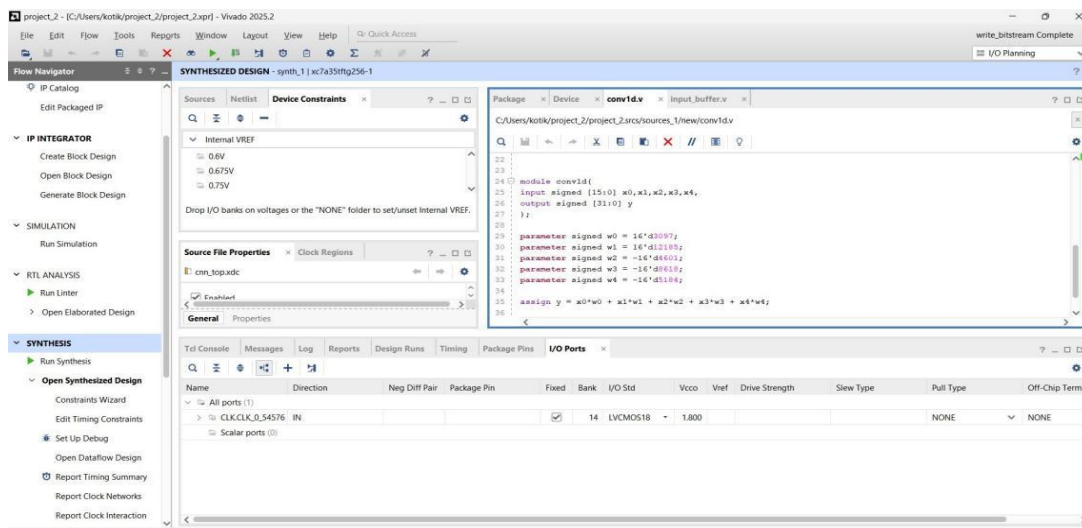
They play a supporting role in preprocessing and analysis.

#### 4.3.3 Matplotlib

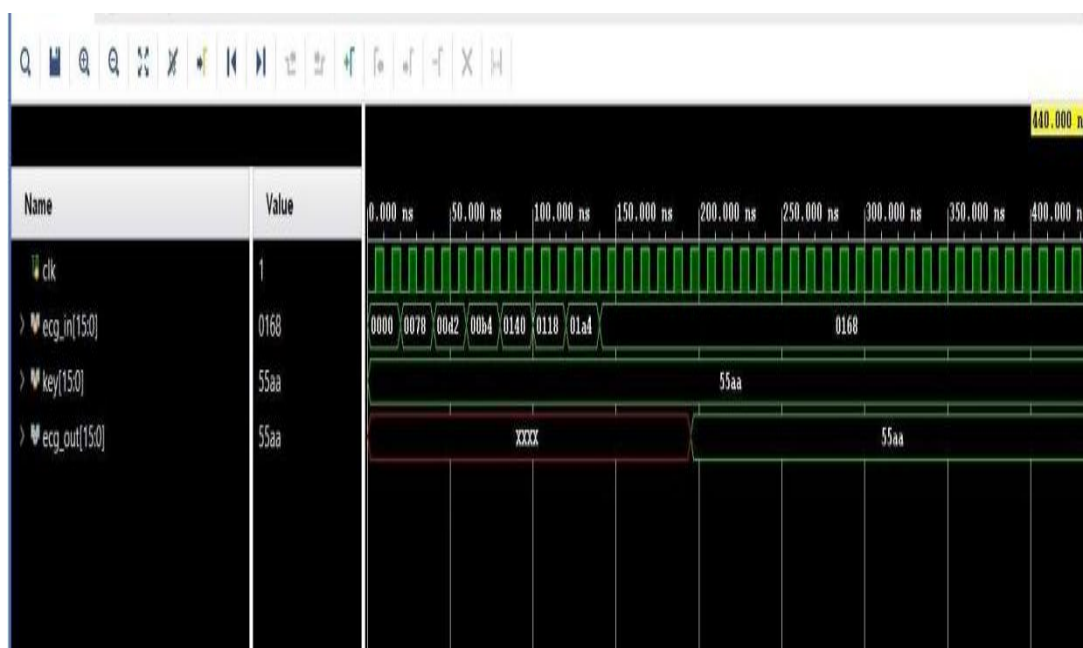
Matplotlib is used for visualizing ECG signals. It helps in plotting:

- Original signal
- Noisy signal
- Denoised signal

Visualization is important for analyzing system performance.



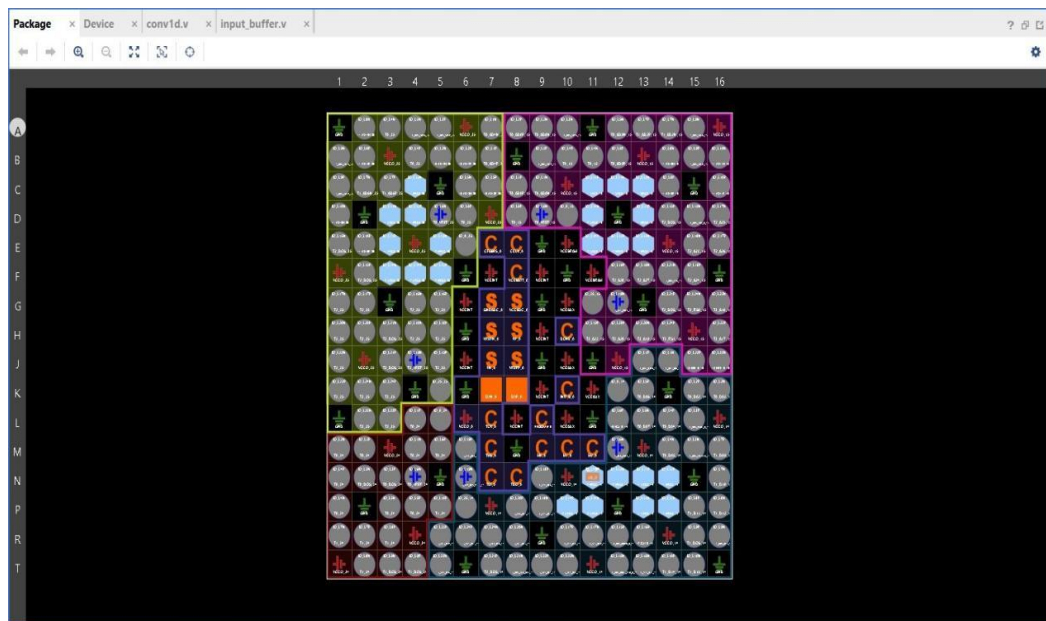
**Fig 4.3 HDL code editor**



**Fig 4.4 Waveform simulation output**

Fig 4.4 shows, the simulation waveform represents the functional verification of the CNN-based denoising system using the testbench. It shows the behavior of key signals such as the clock (clk), input signal (ecg\_in), encryption key (key), and output signal (ecg\_out) over time. The clock signal (clk) is generated as a periodic square wave, which drives the synchronous operation of all modules in the system. The input signal (ecg\_in)

is applied as a sequence of values at different time intervals, representing the sampled input data provided to the system. The encryption key (key) is maintained as a constant value (55AA), which is used for both encryption and decryption processes. Initially, the output signal (ecg\_out) shows undefined values (XXXX), which is expected behavior due to initialization and pipeline delay in the system. As the simulation progresses, the output stabilizes and begins to produce valid values. This delay occurs because the data passes through multiple stages, including input buffering, convolution layers, and activation functions. Once the pipeline is filled, the output reflects the processed data corresponding to the input signal. The waveform confirms that the system is functioning correctly, with proper synchronization between input and output signals. The transition from undefined to valid output demonstrates the internal processing delay, while the steady output values indicate stable operation of the CNN pipeline.



**Fig 4.5 FPGA layout view**

Fig 4.5, represents the FPGA device view after synthesis and implementation of the proposed CNN-based denoising system. It shows how the design is mapped onto different resources available in the FPGA fabric. The device is divided into multiple regions, each containing configurable logic blocks (CLBs), lookup tables (LUTs), flip-flops, and routing resources. The highlighted colored sections indicate the areas where

the design components are placed. These include convolution modules, buffer units, control logic, and memory blocks. The presence of dedicated blocks such as Block RAM is visible, which are used for storing CNN weights loaded through .coe files. The DSP slices are utilized for performing efficient multiply-and-accumulate operations required in convolution layers. These hardware resources enable high-speed arithmetic operations and optimized performance. The distribution of logic across different regions indicates that the design is well-balanced and does not create excessive congestion in any specific area. This ensures efficient routing and stable timing performance. The placement also reflects the modular nature of the design, where different CNN layers and buffers are mapped across the FPGA fabric.

#### **4.4 Advantages and Disadvantages of Vivado**

##### **Advantages**

- Supports large and complex designs
- Integrated IP catalog
- Real-time power and timing analysis
- Graphical block design simplifies SoC creation
- Supports both Verilog and VHDL

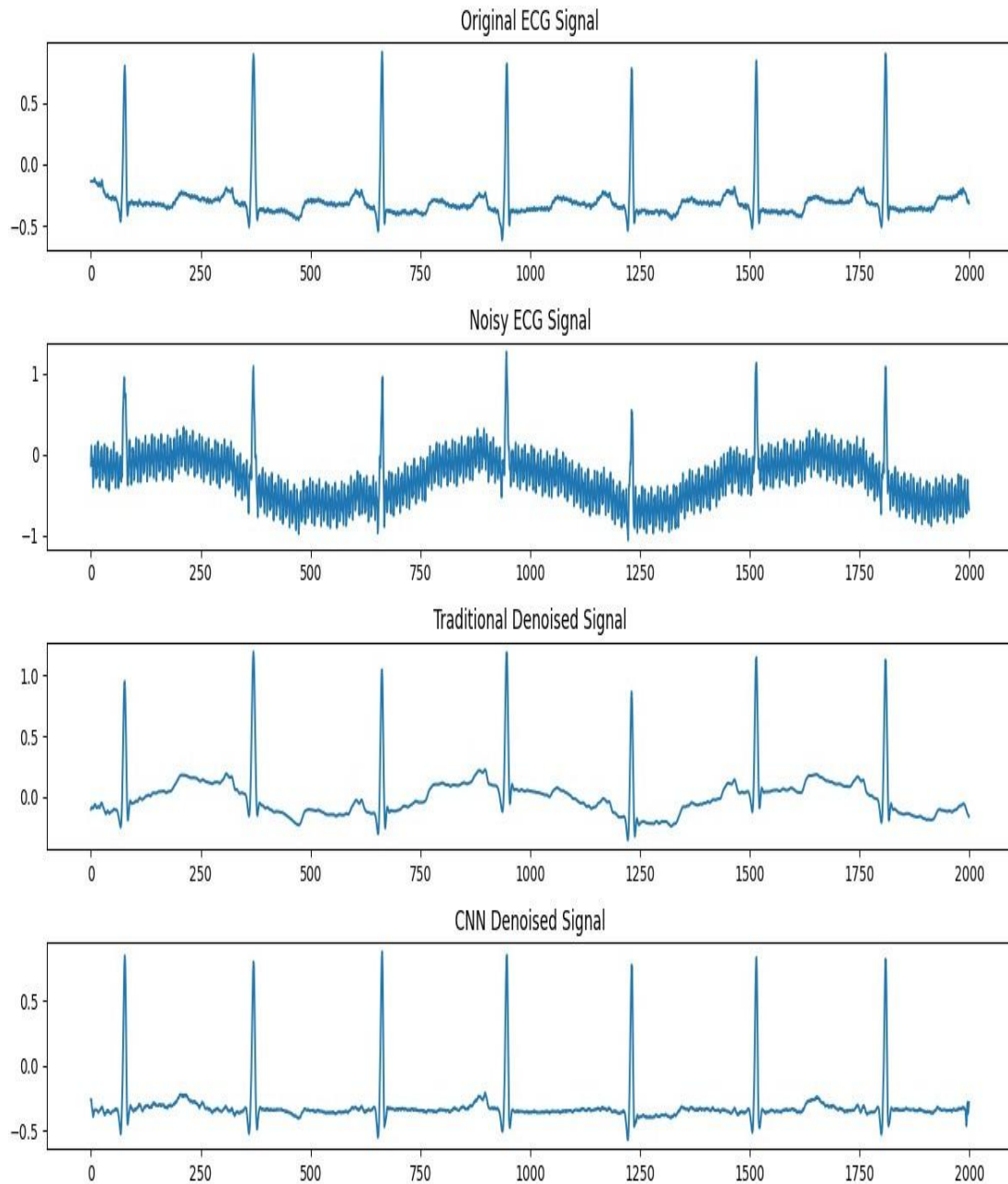
##### **Disadvantages**

- Requires high-performance PC (resource-intensive)
- Learning curve for new users
- Only supports Xilinx FPGAs
- Commercial license is expensive (for full IP access)



## CHAPTER 5

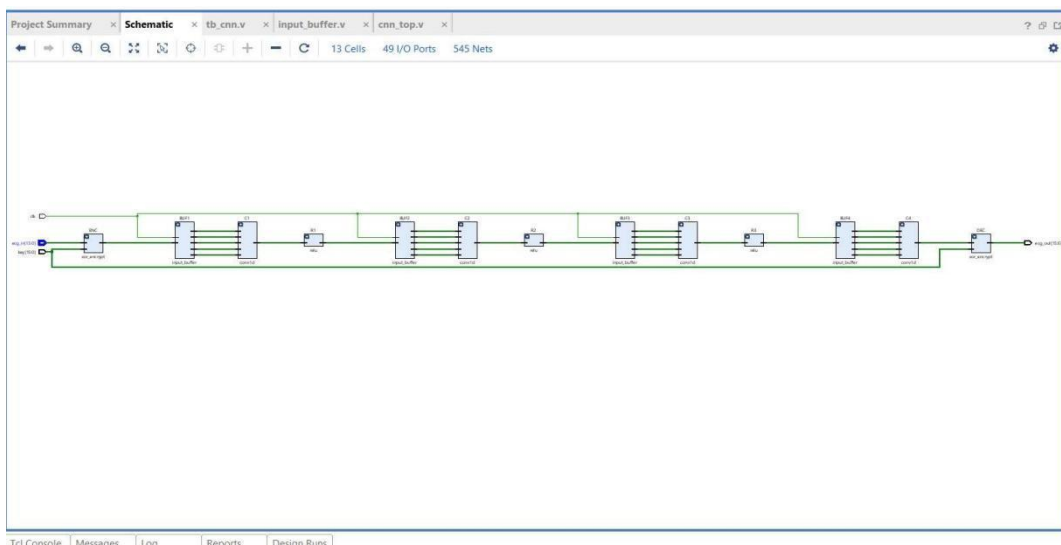
### RESULTS AND DISCUSSION



**Fig 5.1 Signals generated using python**

Fig 5.1, illustrates the comparison between the original signal, noisy signal, traditional denoised signal, and CNN-based denoised signal. The original signal represents the clean signal with all important features intact. The noisy signal shows significant distortion due to

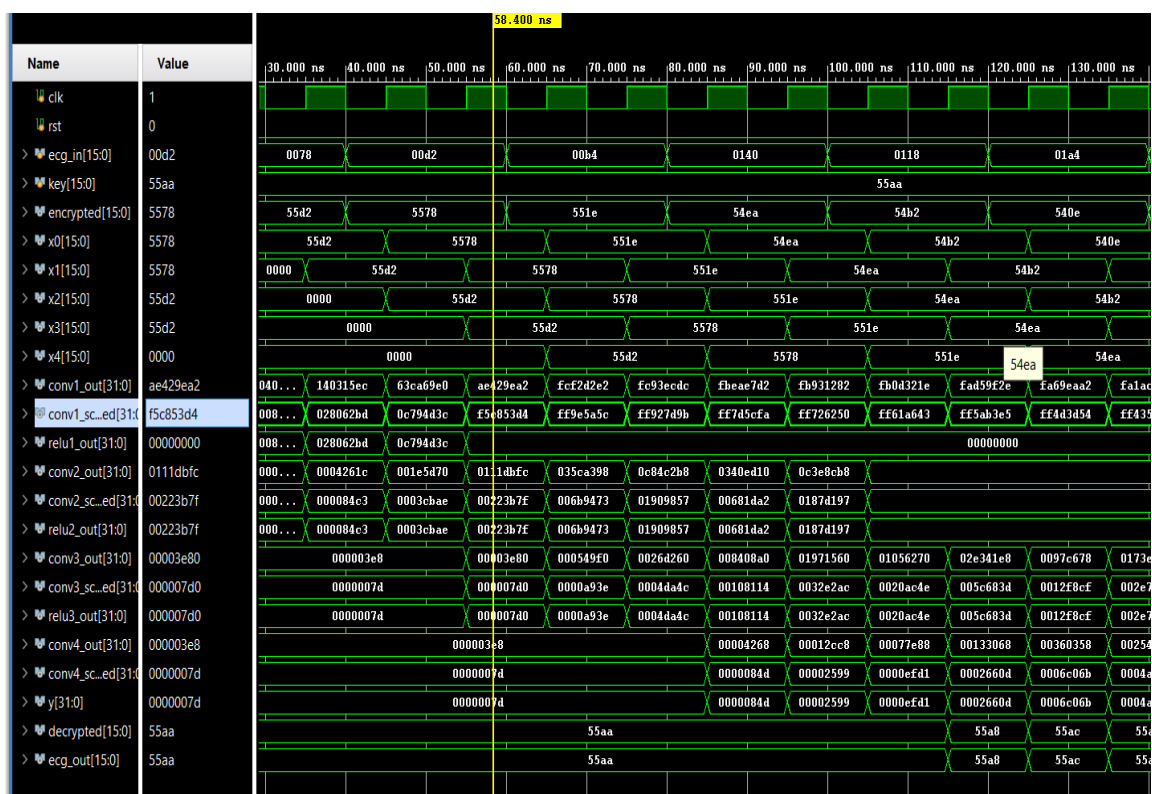
the addition of noise, including high-frequency fluctuations and baseline variations, which obscure the original signal characteristics. The traditional denoised signal demonstrates partial noise removal using conventional filtering techniques. While some noise components are reduced, the signal still contains distortions, and certain important features are not fully preserved. This indicates the limitations of traditional methods in handling complex noise patterns. In contrast, the CNN-based denoised signal shows a significant improvement in signal quality. The noise is effectively removed while preserving the essential features of the original signal. The waveform closely resembles the original signal, indicating the ability of the CNN model to learn and reconstruct clean signal patterns. The effectiveness of the proposed method is further supported by quantitative results. The traditional method achieved a Signal-to-Noise Ratio (SNR) of approximately 0.51 and a Mean Squared Error (MSE) of 0.119. In comparison, the CNN-based approach significantly improved the SNR to 15.89 and reduced the MSE to 0.003, demonstrating superior denoising performance.



**Fig 5.2 Schematic diagram of Convolution and Relu**

Fig 5.2, represents the RTL (Register Transfer Level) schematic of the proposed CNN-based denoising system after synthesis. It illustrates how different modules are connected and how data flows through the system. The design begins with the input signals, which include the input data, encryption key, and clock signal. The input signal is first passed through the XOR encryption module, where it is combined with the key to produce an encrypted output. The encrypted signal is then fed into the input buffer, which operates as a shift register. This buffer generates a sliding window of samples that are required for

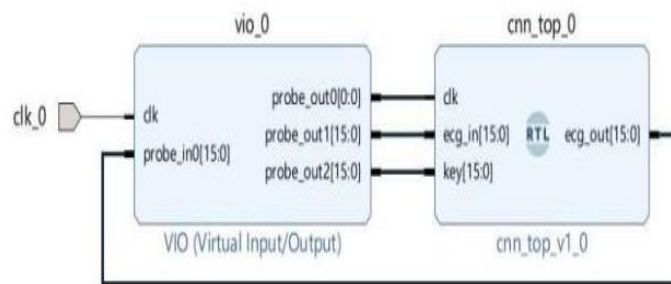
convolution operations. The buffered data is then passed to the first convolution module. The system consists of multiple stages of convolution and activation (ReLU) blocks arranged sequentially. Each convolution module performs multiply-and-accumulate operations using fixed-point weights, and the output is passed through a ReLU activation function to introduce non-linearity. Between these stages, additional buffer modules are used to maintain proper data alignment and synchronization. The outputs from each stage are forwarded to the next stage, forming a pipelined architecture. This sequential flow enables progressive noise reduction and feature extraction across multiple layers. After processing through all convolution layers, the final output is passed through the XOR decryption module, where the original signal structure is recovered using the same key



**Fig 5.3 Simulation Waveform of ECG with Convolution and Relu**

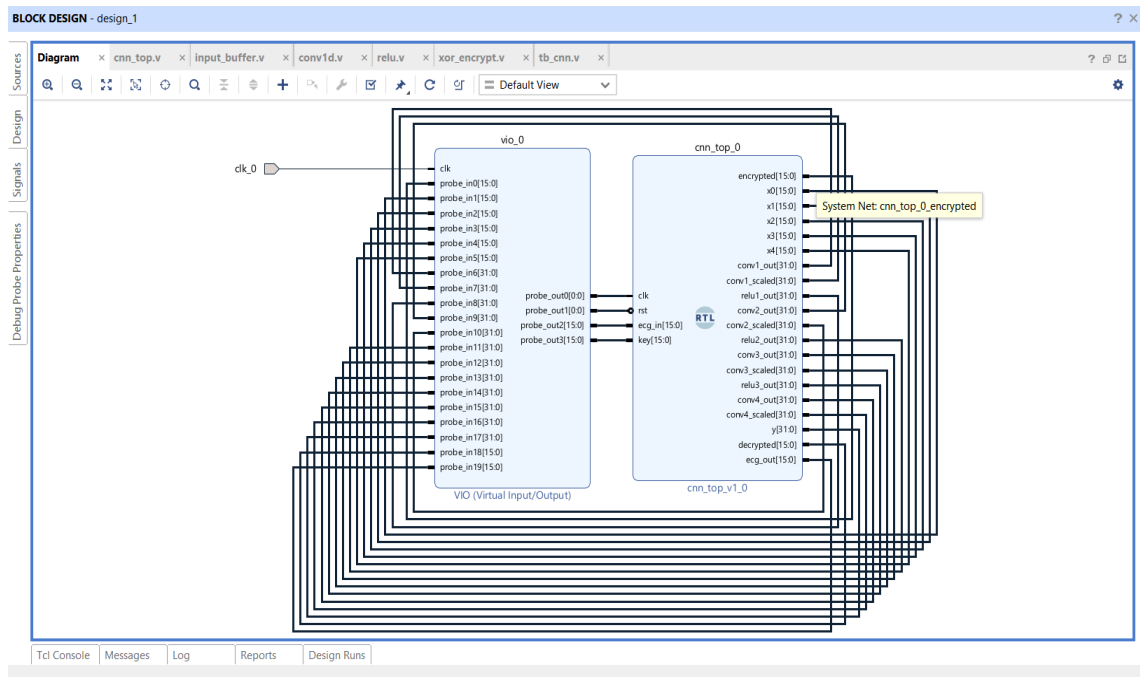
Fig 5.3, represents the detailed internal simulation of the CNN-based denoising system, showing how data flows through different stages of the architecture. The clock signal (clk) drives the entire system and ensures synchronous operation of all modules. The input signal (ecg\_in) is applied as a sequence of values, and the encryption key (key) remains constant at 55AA. The input signal is first processed by the encryption module, producing the signal labeled as encrypted. This confirms that the XOR-based encryption is

functioning correctly. The encrypted signal is then passed through the input buffer, which generates a sliding window of samples (x0, x1, x2, x3, x4). The waveform clearly shows how new samples enter the buffer while older samples shift forward, forming the required window for convolution.



**Fig 5.4 IP Integrator Block**

Fig 5.4, represents the block design of the implemented system in Vivado, illustrating the integration of the CNN processing module with the Virtual Input/Output (VIO) IP core. The design consists of two main blocks: the VIO (Virtual Input/Output) module and the `cnn_top` module, which represents the core processing unit of the system. A clock signal (`clk_0`) is provided to both modules to ensure synchronous operation. The VIO module is used for real-time interaction with the FPGA design. It allows the user to manually provide input signals and observe output signals without requiring external hardware. The signals `ecg_in` and `key` are applied through the VIO as input probes, while the processed signal `ecg_out` is monitored through output probes. The `cnn_top` module receives the input signal and `key` from the VIO module. It performs encryption, buffering, convolution operations, activation functions, and decryption internally to generate the denoised output signal. The interconnections between the VIO and `cnn_top` modules ensure seamless data transfer. The output of the `cnn_top` module is fed back to the VIO, enabling real-time observation and debugging.



**Fig 5.5 IP Integrator block of 4 Convolution layers**

Fig 5.5 shows the integration of the CNN-based denoising module (cnn\_top\_0) with the Virtual Input/Output (VIO) module (vio\_0) on FPGA. The VIO provides input signals such as clock, reset, ECG input, and key, and also allows real-time monitoring of internal signals like encrypted data, sliding window values (x0–x4), convolution outputs, scaled values, ReLU outputs, and final ECG output. The cnn\_top module performs encryption, convolution (MAC operations), activation (ReLU), scaling, and decryption in a pipelined manner. The interconnections between VIO and CNN enable both control and observation of the entire processing flow, confirming correct data propagation and real-time hardware functionality.

| hw_vios                              |          |          |            |          |
|--------------------------------------|----------|----------|------------|----------|
| hw_vio_1                             |          |          |            |          |
| Name                                 | Value    | Activ... | Directi... | VIO      |
| > design_1_i/cnn_top_0_ecg_out[15:0] | [H] 0101 |          | Input      | hw_vio_1 |
| > design_1_i/vio_0_probe_out0[0:0]   | [B] 1    |          | Output     | hw_vio_1 |
| > design_1_i/vio_0_probe_out1[15:0]  | [H] 0110 |          | Output     | hw_vio_1 |
| > design_1_i/vio_0_probe_out2[15:0]  | [H] 0101 |          | Output     | hw_vio_1 |

**Fig 5.6 Hardware Implementation result**

Fig 5.6 represents the hardware debugging interface using the Virtual Input/Output (VIO) core in Vivado Hardware Manager. This interface allows real-time interaction with the

FPGA design by providing input signals and observing output signals directly. The VIO module consists of multiple probes categorized as input and output. The signals labeled as probe\_out represent the inputs provided to the design, while probe\_in represents the outputs observed from the system.

In this implementation:

- The signal vio\_0\_probe\_out1[15:0] is used to provide the input signal.
- The signal vio\_0\_probe\_out2[15:0] represents the encryption key.
- The signal cnn\_top\_0\_ecg\_out[15:0] is the output of the CNN processing module.

The values shown in hexadecimal format (e.g., 0110, 0101) indicate the current data being applied or observed. By modifying the input values through the VIO interface, different test cases can be applied to the system without requiring external hardware. The output values change according to the input, demonstrating that the FPGA design is functioning correctly. This setup allows verification of encryption, buffering, convolution processing, and decryption in real-time.

| Name                                       | Value         | Activ... | Directi... | VIO      |
|--------------------------------------------|---------------|----------|------------|----------|
| >  design_1_i/cnn_top_0_ecg_out[15:0]      | [H] 55AA      |          | Input      | hw_vio_1 |
| >  design_1_i/cnn_top_0_encrypted[15:0]    | [H] 55D2      |          | Input      | hw_vio_1 |
| >  design_1_i/cnn_top_0_relu3_out[31:0]    | [H] 0000_007D |          | Input      | hw_vio_1 |
| >  design_1_i/cnn_top_0_x1[15:0]           | [H] 0000      |          | Input      | hw_vio_1 |
| >  design_1_i/cnn_top_0_x4[15:0]           | [H] 0000      |          | Input      | hw_vio_1 |
| design_1_i/vio_0_probe_out0[0:0]           | [B] 1         |          | Output     | hw_vio_1 |
| design_1_i/vio_0_probe_out1[0:0]           | [B] 0         |          | Output     | hw_vio_1 |
| >  design_1_i/vio_0_probe_out2[15:0]       | [H] 0078      |          | Output     | hw_vio_1 |
| >  design_1_i/cnn_top_0_conv1_out[31:0]    | [H] 040E_3D6A |          | Input      | hw_vio_1 |
| >  design_1_i/cnn_top_0_conv2_out[31:0]    | [H] 0000_03E8 |          | Input      | hw_vio_1 |
| >  design_1_i/cnn_top_0_conv2_scaled[31:0] | [H] 0000_007D |          | Input      | hw_vio_1 |
| >  design_1_i/cnn_top_0_conv3_scaled[31:0] | [H] 0000_007D |          | Input      | hw_vio_1 |
| >  design_1_i/cnn_top_0_conv4_scaled[31:0] | [H] 0000_007D |          | Input      | hw_vio_1 |
| >  design_1_i/cnn_top_0_decrypted[15:0]    | [H] 55AA      |          | Input      | hw_vio_1 |
| >  design_1_i/cnn_top_0_relu1_out[31:0]    | [H] 0081_C7AD |          | Input      | hw_vio_1 |
| >  design_1_i/cnn_top_0_relu2_out[31:0]    | [H] 0000_007D |          | Input      | hw_vio_1 |
| >  design_1_i/cnn_top_0_x0[15:0]           | [H] 55D2      |          | Input      | hw_vio_1 |
| >  design_1_i/cnn_top_0_x2[15:0]           | [H] 0000      |          | Input      | hw_vio_1 |
| >  design_1_i/cnn_top_0_x3[15:0]           | [H] 0000      |          | Input      | hw_vio_1 |
| >  design_1_i/cnn_top_0_y[31:0]            | [H] 0000_007D |          | Input      | hw_vio_1 |
| >  design_1_i/vio_0_probe_out3[15:0]       | [H] 55AA      |          | Output     | hw_vio_1 |
| >  design_1_i/cnn_top_0_conv1_scaled[31:0] | [H] 0081_C7AD |          | Input      | hw_vio_1 |

**Fig 5.7 Hardware Implementation result of 4 Convolution layers**

Fig 5.7 results obtained using the Virtual Input/Output (VIO) interface confirm the successful real-time execution of the designed CNN architecture on hardware. The output signals such as ecg\_out, encrypted, and intermediate convolution outputs are stable and match the expected behavior observed in simulation. The VIO allows dynamic input control

and real-time monitoring of internal signals, demonstrating that the design operates correctly under hardware conditions. Although the output values are stable and valid, the denoising performance is limited due to scaling and weight tuning issues, indicating that further optimization is required for improved accuracy. Overall, the hardware results validate the correctness and successful deployment of the design on FPGA.

The FPGA-based implementation provides: An efficient and high-performance solution for signal denoising. It enables parallel processing of data, resulting in faster computation compared to software-based approaches. The use of fixed-point arithmetic reduces hardware complexity and power consumption while maintaining acceptable accuracy. Additionally, the design ensures low latency and real-time data processing capabilities. Overall, the FPGA implementation offers a reliable, scalable, and resource-efficient platform for implementing CNN-based denoising systems.

## 5.1 PERFORMANCE METRICS

To evaluate the performance of the proposed system, quantitative metrics such as Signal-to-Noise Ratio (SNR) and Mean Squared Error (MSE) are used.

### 5.1.1 Signal-to-Noise Ratio (SNR)

SNR is used to measure the quality of the denoised signal. It indicates how much noise has been removed.

$$SNR = 10 \log_{10} \left( \frac{\sum x(n)^2}{\sum (x(n) - y(n))^2} \right)$$

A higher SNR value indicates better denoising performance.

### 5.1.2 Mean Squared Error (MSE)

MSE measures the difference between the original and denoised signals.

$$MSE = \frac{1}{N} \sum (x(n) - y(n))^2$$

A lower MSE value indicates higher accuracy.

### 5.1.3 Observations

- CNN method produces higher SNR compared to traditional methods
- CNN method results in lower MSE
- FPGA implementation maintains similar performance as software

## 5.2 PERFORMANCE COMPARISON

Traditional SNR: 0.5105442567798956

Traditional MSE: 0.11967297127714789

Epoch: 0 Loss: 0.0039465283043682575

Epoch: 1 Loss: 0.002971129259094596

NN SNR: 15.89112627819362

CNN MSE: 0.003027325717030443

| Method             | SNR (dB) | MSE       | Observation      |
|--------------------|----------|-----------|------------------|
| Noisy Signal       | Very Low | Very High | Highly distorted |
| Traditional Filter | Moderate | Moderate  | Loss of features |
| CNN Method         | High     | Low       | Best performance |

**Table 5.1 Performance Comparison**

## 5.3 DISCUSSION

The results obtained from the proposed system clearly demonstrate the effectiveness of the CNN-based approach for ECG signal denoising. The CNN model is able to learn complex noise patterns and remove them without affecting important signal features.

Traditional filtering methods, although simple, are limited in their ability to handle multiple noise sources. They often result in distortion of the signal, which reduces the accuracy of diagnosis. In contrast, the CNN model provides adaptive and data-driven noise removal.

The FPGA implementation further enhances the system by enabling real-time processing. The use of parallel architecture allows faster computation, while fixed-point implementation ensures efficient use of hardware resources. The inclusion of encryption adds an additional layer of security, making the system suitable for handling sensitive medical data. This is particularly important in modern healthcare systems where data privacy is a major concern.

Overall, the proposed system successfully combines deep learning and hardware acceleration to achieve high-performance ECG denoising. The results indicate that the system is reliable, efficient, and suitable for real-world biomedical applications.



## **5.4 LIMITATIONS OF PROPOSED SYSTEM**

### **1. Limited CNN Complexity**

The implemented CNN model is simple due to FPGA resource constraints, limiting performance for complex noise patterns.

### **2. Fixed-Point Precision Loss**

Conversion to Q1.15 format introduces quantization errors, reducing accuracy compared to floating-point models.

### **3. No On-Chip Training**

The system supports only inference using pre-trained weights and cannot adapt to new data in real time.

### **4. Single-Channel Processing**

The design handles only single input signals and does not support multi-channel data.

### **5. Basic Encryption Method**

XOR-based encryption provides low-level security and is not suitable for high-security applications.

### **6. FPGA Resource Constraints**

Limited LUTs, DSPs, and BRAM restrict implementation of deeper or more complex CNN architectures.

### **7. Pipeline Latency**

Initial delay occurs due to buffering and pipelined processing stages.

### **8. Manual Weight Integration**

Weight loading using .coe files requires preprocessing and lacks flexibility for dynamic updates.

### **9. Limited User Interface**

Interaction through VIO is not user-friendly for practical deployment.

### **10. Scalability Issues**

Extending the design to larger datasets or higher-resolution signals requires further optimization.

## **CHAPTER 6**

### **CONCLUSION AND FUTURE SCOPE**

#### **6.1 CONCLUSION**

This project presents the design and implementation of a CNN-based signal denoising system using FPGA. The proposed approach successfully combines deep learning techniques with hardware acceleration to achieve efficient and reliable performance.

The Convolutional Neural Network was trained to effectively remove noise while preserving important signal features. The trained model parameters were converted into fixed-point representation and implemented on FPGA using Verilog HDL. The use of modules such as input buffers, convolution units, activation functions, and encryption/decryption blocks ensured a structured and modular design.

Simulation results verified the correctness of the system, while hardware implementation demonstrated real-time functionality using VIO. The performance improvement is evident from the significant increase in Signal-to-Noise Ratio (SNR) and reduction in Mean Squared Error (MSE) compared to traditional methods.

Overall, the FPGA-based implementation provides high speed, low latency, and efficient parallel processing. The project demonstrates that CNN-based denoising can be effectively realized on hardware with optimized resource utilization and improved performance.

#### **6.2 Advantages**

- 1. High Denoising Accuracy**

The CNN model effectively removes noise while preserving important signal features, resulting in improved signal quality.

- 2. Improved Performance Metrics**

The system achieves higher Signal-to-Noise Ratio (SNR) and lower Mean Squared Error (MSE) compared to conventional methods.

- 3. Parallel Processing Capability**

FPGA enables parallel execution of operations, significantly increasing processing speed.

**4. Low Latency**

The pipelined architecture ensures faster data processing with minimal delay.

**5. Efficient Hardware Utilization**

The use of fixed-point arithmetic (Q1.15 format) reduces resource usage and power consumption.

**6. Scalable Architecture**

The modular design allows easy expansion to more layers or advanced models.

**7. Real-Time Processing Support**

The system can process continuous data streams efficiently.

**8. Secure Data Processing**

XOR-based encryption ensures that the input data remains protected during processing.

**9. Reduced Computational Complexity**

Hardware implementation minimizes the need for complex software computations.

**10. Easy Debugging and Testing**

Tools like VIO enable real-time monitoring and debugging directly on FPGA.

## **6.3 FUTURE SCOPE**

**1. Advanced Deep Learning Models**

Implement deeper CNN architectures or hybrid models to improve denoising accuracy.

**2. Model Optimization Techniques**

Apply pruning and quantization-aware training to reduce hardware resource utilization and improve efficiency.

**3. Multi-Channel Signal Processing**

Extend the system to support multiple input channels for handling complex signal data.

**4. Real-Time Streaming Applications**

Enable continuous data processing for real-time monitoring systems.

**5. Integration with External Devices**

Connect with sensors and communication modules for practical deployment in healthcare systems.

**6. Enhanced Security Mechanisms**

Incorporate advanced encryption techniques to improve data protection.

**7. Scalability for Larger Data**

Modify the system to support larger datasets and higher-resolution signals.

**8. Implementation on Advanced Hardware**

Deploy the design on advanced FPGA platforms or SoC devices for improved performance.

**9. Extension to Other Biomedical Signals**

Apply the system to signals such as EEG and EMG for broader applicability.

**10. User-Friendly Interface Development**

Develop a graphical interface for better visualization, control, and ease of use.

## REFERENCES

- [1] V. Chauhan and S. Changlani, “Signal Processing in VLSI Circuits Using Neural Networks,” *International Journal of Creative Research Thoughts (IJCRT)*, vol. 13, no. 9, 2025.
- [2] G. R. Thippeswamy et al., “AI-Based Machine Learning Algorithm in VLSI Technology,” *International Journal of Research in Engineering and Science (IJRES)*, 2023.
- [3] S. Tejesh Reddy et al., “AI-Driven Signal Processing for Noise Reduction,” *International Journal of Creative Research Thoughts (IJCRT)*, 2022.
- [4] K. Zhang et al., “Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising,” *IEEE Transactions on Image Processing*, vol. 26, no. 7, pp. 3142– 3155, 2017.
- [5] M. M. Fahmy et al., *FPGA-Based Acceleration of Deep Learning Algorithms*, Springer, 2019.
- [6] P. Laguna et al., “A Database for Evaluation of Algorithms for Measurement of QT and Other Waveform Intervals in the ECG,” *Computers in Cardiology*, pp. 673–676, 1997.
- [7] A. Krizhevsky et al., “ImageNet Classification with Deep Convolutional Neural Networks,” *Advances in Neural Information Processing Systems*, 2012.
- [8] S. T. Reddy, S. S. Ogety, and N. V. Annapurna, “AI-Driven Signal Processing: Improving Communication System with Machine Learning-Based Noise Reduction,” in *2nd International Conference on Advance Computing and Innovative Technologies in Engineering (ICACITE)*, 2022.
- [9] N. Mashhadi, A. Z. Khuzani, M. Heidari, and D. Khaledyan, “Deep Learning Denoising for EOG Artifacts Removal from EEG Signals,” in *IEEE Global Humanitarian Technology Conference*, 2020.
- [10] A. Roy, A. P., L. Tomy, and M. Rajeswari, “Recent Study on Image Denoising using Deep CNN Techniques,” in *International Conference on Advanced Computing and Communication Systems (ICACCS)*, 2021.
- [11] A. Shekhar, S. Hota, P. Mahalakshmi, and A. Sharmila, “Development of Signal

Processing Algorithms for De-noising of ECG Signals in Wearable Systems and Testing for Arrhythmia using CNN Model,” in Innovations in Power and Advanced Computing Technologies (i-PACT), 2021.

[12] X. Liu, Y. Jiang, H. Wang, and K. Liu, “Research on Digital Signal Denoising Techniques based on Convolutional Neural Networks,” in International Conference on Electronic Information Engineering and Computer Technology (EIECT), 2024.

[13] A. Shukla and I. Izharuddin, “Denoising ECG Signals and their Analysis using Hybrid Deep Learning Model,” in International Conference on Disruptive Technologies (ICDT), 2023.

## APPENDIX

### Source Code

```
import wfdb
import numpy as np
import matplotlib.pyplot as plt # Load record
record = wfdb.rdrecord('100', pn_dir='mitdb') signal = record.p_signal[:, 0]
signal = signal[:20000]
fs = record.fs
signal = signal / np.max(np.abs(signal)) t = np.arange(len(signal)) / fs
# Powerline Noise
powerline_noise = 0.2 * np.sin(2 * np.pi * 50 * t)
# Baseline wander
baseline_wander = 0.3 * np.sin(2 * np.pi * 0.5 * t)
# EMG noise (band-limited)
emg_noise = 0.1 * np.sin(2 * np.pi * 120 * t)
noisy_signal = signal + powerline_noise + baseline_wander + emg_noise signal
= signal[:20000]
from scipy.signal import iirnotch, filtfilt, butter
# Notch filter
b_notch, a_notch = iirnotch(50, 30, fs)
filtered = filtfilt(b_notch, a_notch, noisy_signal)
# Bandpass filter (0.5Hz – 40Hz)
b_band, a_band = butter(4, [0.5/(fs/2), 40/(fs/2)], btype='band')
traditional_denoised = filtfilt(b_band, a_band, filtered)
def calculate_snr(clean, denoised):
    noise = clean – denoised
    snr = 10 * np.log10(np.sum(clean**2) / np.sum(noise**2))
    return snr
def calculate_mse(clean, denoised):
    return np.mean((clean - denoised)**2)
snr_trad = calculate_snr(signal, traditional_denoised)
mse_trad = calculate_mse(signal, traditional_denoised)
```

```

print("Traditional SNR:", snr_trad)
print("Traditional MSE:", mse_trad)
import torch
from torch import nn
from torch.utils.data import Dataset, DataLoader class ECGDataset(Dataset):
def __init__(self, noisy, clean):
self.noisy = torch.tensor(noisy, dtype=torch.float32)
self.clean = torch.tensor(clean, dtype=torch.float32)
def __len__(self):
return len(self.noisy) - 256 def __getitem__(self, idx):
return (self.noisy[idx:idx+256], self.clean[idx:idx+256])
dataset = ECGDataset(noisy_signal, signal)
loader=DataLoader(dataset,batch_size=128,shuffle=True)class CNN_Denoiser(nn.Module):
def __init__(self): super().__init__()
self.net = nn.Sequential( nn.Conv1d(1, 16, 5, padding=2), nn.ReLU(),
nn.Conv1d(16, 32, 5, padding=2), nn.ReLU(),
nn.Conv1d(32, 16, 5, padding=2), nn.ReLU(),
nn.Conv1d(16, 1, 5, padding=2)
)def forward(self, x): return self.net(x)
model = CNN_Denoiser() criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001) for epoch in range(2):
for noisy_batch, clean_batch in loader:
noisy_batch = noisy_batch.unsqueeze(1)
clean_batch = clean_batch.unsqueeze(1)
output = model(noisy_batch)
loss = criterion(output, clean_batch)
optimizer.zero_grad()
loss.backward()
optimizer.step()
print("Epoch:", epoch, "Loss:", loss.item()) with torch.no_grad():
test_input = torch.tensor(noisy_signal [:2000],
dtype=torch.float32).unsqueeze(0).unsqueeze(0)
cnn_output = model(test_input).squeeze().numpy()

```



```

snr_cnn = calculate_snr(signal[:2000],
cnn_output) mse_cnn = calculate_mse(signal[:2000], cnn_output)
print("CNN SNR:", snr_cnn)
print("CNN MSE:", mse_cnn) plt.figure(figsize=(12,8))
plt.subplot(4,1,1)
plt.plot(signal[:2000])
plt.title("Original ECG Signal")
plt.subplot(4,1,2)
plt.plot(noisy_signal[:2000])
plt.title("Noisy ECG Signal")
plt.subplot(4,1,3)
plt.plot(traditional_denoised[:2000])
plt.title("Traditional Denoised Signal")
plt.subplot(4,1,4)
plt.plot(cnn_output[:2000])
plt.title("CNN Denoised Signal")
plt.tight_layout()
plt.show()
plt.subplot(4,1,4)
plt.plot(cnn_output[:2000])
plt.title("CNN Denoised Signal")
plt.tight_layout()
plt.show()
np.savetxt("cnn_denoised_signal.txt", cnn_output)
print("CNN denoised signal saved")
def float_to_fixed(x, q=15): scale = 2**q
return np.round(x * scale).astype(np.int16)
def fixed_to_float(x, q=15):
return x / (2**q)
fixed_weights = {}
print("\nModel Weights Info:")
for name, param in model.state_dict().items():
weights = param.detach().numpy()

```

```

print(name, weights.shape)
np.savetxt(name.replace(".", "_") + "_float.txt",
weights.flatten()) # Convert to fixed
fixed = float_to_fixed(weights) # Save fixed weights
np.savetxt(name.replace(".", "_") + "_Q15.txt", fixed.flatten(), fmt='%d')
fixed_weights[name] = fixed
print("\nFloating and Fixed-point weights saved.")
print("Weights converted to Q1.15 format")
def adaptive_q_format(tensor, total_bits=16):
    max_val = np.max(np.abs(tensor))
    integer_bits = int(np.ceil(np.log2(max_val + 1e-6)))
    fractional_bits = total_bits - 1 - integer_bits
    scale = 2**fractional_bits
    fixed = np.round(tensor * scale).astype(np.int16)
    return fixed, fractional_bits
for name, param in model.state_dict().items():
    print(name, param.shape)
    for name, param in model.state_dict().items():
        np.savetxt(name.replace(".", "_") + "_float.txt", param.numpy().flatten())
    def float_to_fixed(x, q=15): scale = 2**q
    return np.round(x*scale).astype(np.int16)
    for name, param in model.state_dict().items():
        fixed = float_to_fixed(param.numpy())
        filename = name.replace(".", "_") + "_Q15.txt"
        np.savetxt(filename, fixed.flatten(), fmt='%d')
    example = list(model.state_dict().values())[0].numpy().flatten()[10]
    fixed_example = float_to_fixed(example)
    print("Floating values:", example)
    print("Fixed Q1.15 values:", fixed_example)
    print("Fixed point weights saved")
    def export_to_coe(weights, filename): with open(filename, "w") as f:
        f.write("memory_initialization_radix=10;\n")
        f.write("memory_initialization_vector=\n")

```

```

flat = weights.flatten()
for val in flat[:-1]: f.write(str(val) + ",\n")
f.write(str(flat[-1]) + ";")
export_to_coe(fixed_weights['net.0.weight'], "conv1_weights.coe")

```

```

`timescale 1ns/1p module tb_cnn; reg clk;
reg [15:0] ecg_in;
reg [15:0] key;
wire [15:0] ecg_out; cnn_top DUT(
.clk(clk),
.ecg_in(ecg_in),
.key(key),
.ecg_out(ecg_out)
);
initial
begin clk=0;
forever #5 clk=~clk; end /* stimulus */
initial begin
key = 16'h55AA; ecg_in = 0;
#100 ecg_in = 120;
#100 ecg_in = 210;
#100 ecg_in = 180;
#100 ecg_in = 320;
#100 ecg_in = 280;
#100 ecg_in = 420;
#100 ecg_in = 360;
#200 $finish;
end
endmodule

```

```

`timescale 1ns/1ps
module cnn_top( input clk,
input [15:0] ecg_in,

```

```

input [15:0] key,
output [15:0] ecg_out
);
wire [15:0] encrypted;
wire [15:0] decrypted;
wire signed [15:0] x0,x1,x2,x3,x4;
wire signed [31:0] conv1_out;
wire signed [31:0] relu1_out;
wire signed [31:0] conv2_out;
wire signed [31:0] relu2_out;
wire signed [31:0] conv3_out;
wire signed [31:0] relu3_out;
wire signed [31:0] conv4_out;
wire signed [31:0] y;
xor_encrypt ENC(
.data_in(ecg_in),
.key(key),
.data_out(encrypted)
); /// INPUT BUFFER///
input_buffer BUF1(
.clk(clk),
.rst(rst),
.sample_in(encrypted),
.x0(x0),
.x1(x1),
.x2(x2),
.x3(x3),
.x4(x4)
); /// CNN LAYER1///
conv1d C1(
.x0(x0),
.x1(x1),
.x2(x2),

```

```

.x3(x3),
.x4(x4),
.y(conv1_out)
);
relu R1(
.data_in(conv1_out),
.data_out(relu1_out)
); /// BUFFER2///
wire signed [15:0] b1_0,b1_1,b1_2,b1_3,b1_4;
input_buffer BUF2(
.clk(clk),
.sample_in(relu1_out[31:16]),
.x0(b1_0),
.x1(b1_1),
.x2(b1_2),
.x3(b1_3),
.x4(b1_4)
); /// CNN LAYER2///
conv1d C2(
.x0(b1_0),
.x1(b1_1),
.x2(b1_2),
.x3(b1_3),
.x4(b1_4),
.y(conv2_out)
);
relu R2(
.data_in(conv2_out),
.data_out(relu2_out)
); ///BUFFER3///
wire signed [15:0] b2_0,b2_1,b2_2,b2_3,b2_4;
input_buffer BUF3(
.clk(clk),

```

```

.sample_in(relu2_out[31:16]),
.x0(b2_0),
.x1(b2_1),
.x2(b2_2),
.x3(b2_3),
.x4(b2_4)
);    /// CNN LAYER3///
conv1d C3(
.x0(b2_0),
.x1(b2_1),
.x2(b2_2),
.x3(b2_3),
.x4(b2_4),
.y(conv3_out)
);
relu R3(
.data_in(conv3_out),
.data_out(relu3_out)
);    /// BUFFER4///
wire signed [15:0] b3_0,b3_1,b3_2,b3_3,b3_4;
input_buffer BUF4(
.clk(clk),
.sample_in(relu3_out[31:16]),
.x0(b3_0),
.x1(b3_1),
.x2(b3_2),
.x3(b3_3),
.x4(b3_4)
);    /// CNN LAYER4///
conv1d C4(
.x0(b3_0),
.x1(b3_1),
.x2(b3_2),

```

```

.x3(b3_3),
.x4(b3_4),
.y(conv4_out)
);
assign y = conv4_out;
/// DECRYPTION///
xor_encrypt DEC(
.data_in(y[31:16]),
.key(key),
.data_out(decrypted)
);
assign ecg_out = decrypted;
endmodule

```

```

module conv1d(
input signed [15:0] x0,x1,x2,x3,x4,
output signed [31:0] y
);
parameter signed w0 = 16'd3097;
parameter signed w1 = 16'd12185;
parameter signed w2 = -16'd4601;
parameter signed w3 = -16'd8618;
parameter signed w4 = -16'd5184;
assign y = x0*w0 + x1*w1 + x2*w2 + x3*w3 + x4*w4;
endmodule

```

```

module input_buffer( input clk,
input rst,
input signed [15:0] sample_in,
output reg signed [15:0] x0,x1,x2,x3,x4
);
always @(posedge clk or posedge rst)
begin

```

```

if(rst) begin
x0 <= 0;
x1 <= 0;
x2 <= 0;
x3 <= 0;
x4 <= 0;
end
else begin
x4 <= x3; x3 <= x2; x2 <= x1; x1 <= x0;
x0 <= sample_in;
end
end
endmodule

```

```

module relu(
input signed [31:0] data_in,
output signed [31:0] data_out
);
assign data_out = (data_in > 0) ? data_in : 0;
endmodule

module xor_encrypt( input [15:0] data_in,
input [15:0] key,
output [15:0] data_out
);
assign data_out = data_in ^ key;
endmodule

```

```

module cnn_controller( input clk,
output reg conv1_en,
output reg conv2_en,
output reg conv3_en,
output reg conv4_en
);

```



```

reg [2:0] state = 0;
always @(posedge clk) begin case(state)
0: begin
conv1_en <= 1;
conv2_en <= 0;
conv3_en <= 0;
conv4_en <= 0;
state <= 1; end
1: begin
conv1_en <= 0;
conv2_en <= 1;
state <= 2; end
2: begin
conv2_en <= 0;
conv3_en <= 1;
state <= 3; end
3: begin
conv3_en <= 0;
conv4_en <= 1;
state <= 0; end
endcase
end
endmodule

```

```

module conv_layer1( input clk,
input signed [15:0] x0,x1,x2,x3,x4,
output signed [31:0] y
);
reg [6:0] addr;
wire signed [15:0] weight;
blk_mem_gen_0 conv1_weights(
.clka(clk),
.addra(addr),

```

```

.douta(weight)
);
reg signed [15:0] w0,w1,w2,w3,w4;
reg [2:0] state=0;
always @(posedge clk) begin
case(state)
0: begin addr<=0; state<=1; end
1: begin w0<=weight; addr<=1; state<=2; end
2: begin w1<=weight; addr<=2; state<=3; end
3: begin w2<=weight; addr<=3; state<=4; end
4: begin w3<=weight; addr<=4; state<=5; end
5: begin w4<=weight; state<=6; end
endcase
end
conv1d CONV(x0,x1,x2,x3,x4,w0,w1,w2,w3,w4,y);
endmodule

```

```

module conv_layer2( input clk,
input signed [31:0] in_data,
output signed [31:0] y
);
reg [11:0] addr;
wire signed [15:0] weight;
blk_mem_gen_1 conv2_weights(
.clka(clk),
.addra(addr),
.douta(weight)
);
reg signed [15:0] w0,w1,w2,w3,w4;
wire signed [15:0] x;
assign x = in_data[31:16];
reg [2:0] state=0;
always @(posedge clk)

```

```

begin
case(state)
0: begin addr<=0; state<=1; end
1: begin w0<=weight; addr<=1; state<=2; end
2: begin w1<=weight; addr<=2; state<=3; end
3: begin w2<=weight; addr<=3; state<=4; end
4: begin w3<=weight; addr<=4; state<=5; end
5: begin w4<=weight; state<=6; end
endcase
end
conv1d CONV(x,x,x,x,x,w0,w1,w2,w3,w4,y);
endmodule

```

```

module conv_layer3( input clk,
input signed [31:0] in_data,
output signed [31:0] y
);
reg [11:0] addr;
wire signed [15:0] weight;
blk_mem_gen_2 conv3_weights(
.clka(clk),
.addra(addr),
.douta(weight)
);
reg signed [15:0] w0,w1,w2,w3,w4;
wire signed [15:0] x;
assign x = in_data[31:16];
reg [2:0] state=0;
always @(posedge clk)
begin
case(state)
0: begin addr<=0; state<=1; end
1: begin w0<=weight; addr<=1; state<=2; end

```

```

2: begin w1<=weight; addr<=2; state<=3; end
3: begin w2<=weight; addr<=3; state<=4; end
4: begin w3<=weight; addr<=4; state<=5; end
5: begin w4<=weight; state<=6; end
endcase end
conv1d CONV(x,x,x,x,x,w0,w1,w2,w3,w4,y);
endmodule

```

```

module conv_layer4( input clk,
input signed [31:0] in_data,
output signed [31:0] y
);
reg [6:0] addr;
wire signed [15:0] weight;
blk_mem_gen_3 conv4_weights(
.clka(clk),
.addra(addr),
.douta(weight));
reg signed [15:0] w0,w1,w2,w3,w4;
wire signed [15:0] x;
assign x = in_data[31:16];
reg [2:0] state=0;
always @(posedge clk) begin
case(state)
0: begin addr<=0; state<=1; end
1: begin w0<=weight; addr<=1; state<=2; end
2: begin w1<=weight; addr<=2; state<=3; end
3: begin w2<=weight; addr<=3; state<=4; end
4: begin w3<=weight; addr<=4; state<=5; end
5: begin w4<=weight; state<=6; end
endcase end
conv1d CONV(x,x,x,x,x,w0,w1,w2,w3,w4,y);
endmodule

```